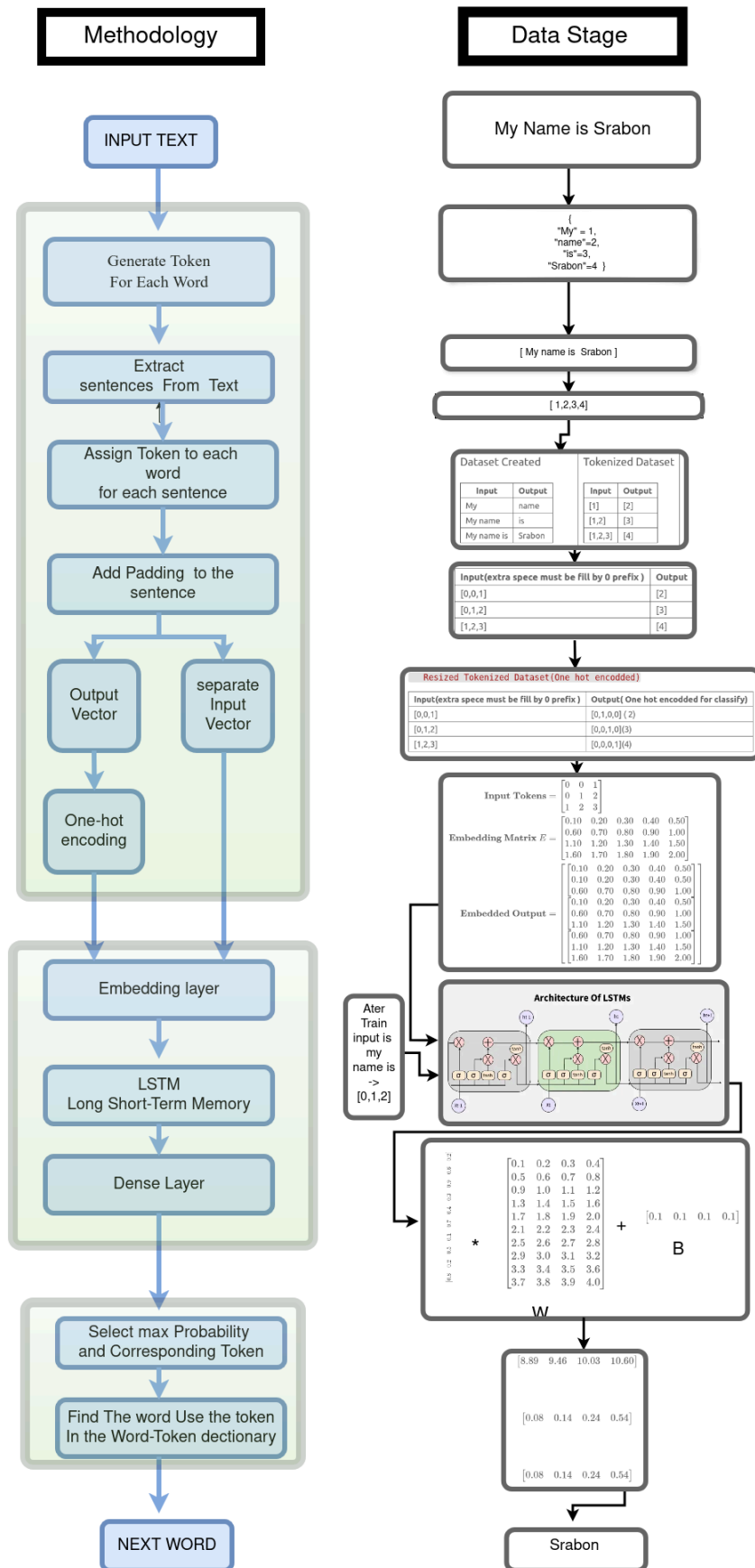


LSTM-Based Next Word Prediction

From Raw Text to Intelligent Word Generation



Project Overview

This project demonstrates the complete implementation of an **LSTM (Long Short-Term Memory)** based **Next Word Prediction** model developed from scratch. Rather than only training a neural network, the notebook explains **every mathematical and preprocessing step** involved in transforming raw text into meaningful predictions.

The objective is to illustrate the entire NLP pipeline—from tokenization and sequence generation to embedding, LSTM learning, dense layer computation, probability estimation, and final word prediction.

Methodology

The complete workflow is illustrated in the methodology diagram above.

1. Text Preprocessing

- Input raw text corpus
- Extract sentences
- Generate a vocabulary
- Assign a unique integer token to every word

2. Sequence Generation

- Convert each sentence into numerical token sequences
- Create multiple input-output training pairs
- Apply zero-padding to produce fixed-length input vectors

3. Label Encoding

- Convert output words into **One-Hot Encoded** vectors for supervised learning.

4. Embedding Layer

- Transform sparse token IDs into dense continuous vector representations.
- Learn semantic relationships between words during training.

5. LSTM Network

- Process embedded sequences through LSTM cells.
- Capture contextual and long-term dependencies between words.

6. Dense Layer

- Perform matrix multiplication

$$[\mathbf{z} = \mathbf{h}W + \mathbf{b}]$$

where

- (h) = LSTM hidden state
- (W) = Dense layer weights
- (b) = Bias vector

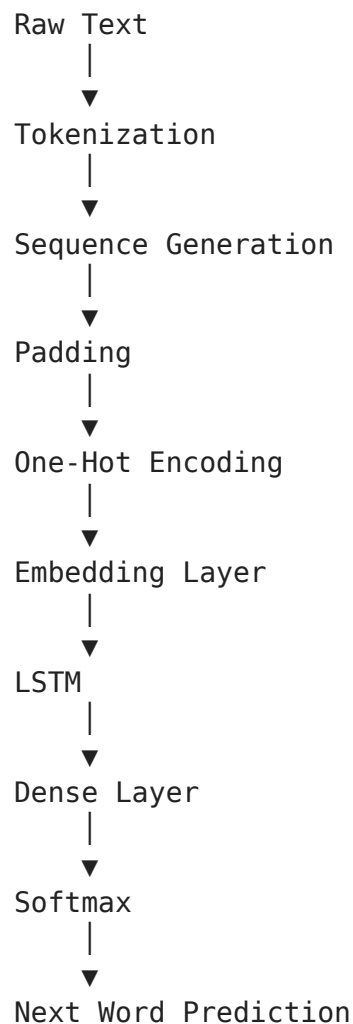
7. Softmax Prediction

Convert logits into probabilities

$$[P(y_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}]$$

The token with the highest probability becomes the predicted next word.

Pipeline Summary



Key Learning Objectives

- ✓ Understand NLP preprocessing from first principles
 - ✓ Visualize how tokenization creates numerical datasets
 - ✓ Learn how embedding converts integers into dense vectors
 - ✓ Understand LSTM memory flow
 - ✓ Follow Dense layer computations mathematically
 - ✓ Interpret Softmax probabilities for next-word prediction
-

Technologies Used

- Python
 - NumPy
 - TensorFlow / Keras
 - LSTM
 - Embedding Layer
 - One-Hot Encoding
 - Jupyter Notebook
-

```
In [1]: text=""" geeksforgeeks

Courses
Tutorials
Interview Prep

DSA
Practice Problems
C
C++
Java
Python
JavaScript
Data Science
Machine Learning
Courses
Linux
DevOps

Introduction to Long Short Term Memory
Last Updated : 18 May, 2026

Long Short-Term Memory (LSTM) is an improved version of the Recurrent Neural

Handles Long Term Dependencies: Remembers information for longer sequence
```

Memory Cell: Stores and updates important information over time
 Better than RNN: Overcomes short term memory limitations
 Applications: Used in language translation, speech recognition and time

introduction_to_lstm.webp
 Problem with Long-Term Dependencies in RNN

RNNs are designed to handle sequential data by using a hidden state that stores

Vanishing Gradient: When training a model over time, the gradients which
 Exploding Gradient: Sometimes gradients can grow too large causing insta

LSTM Architecture

LSTM (Long Short-Term Memory) architecture is designed to learn long-term de
 Main Gates in LSTM

Input Gate: Decides which new information should be added to the memory
 Forget Gate: Determines which information should be removed from the mem
 Output Gate: Controls which information from the memory cell is passed t

Working of LSTM

LSTM consists of a repeating chain like structure with memory cells and gati
 gate_of_lstm
 LSTM Model

Information is retained by the cells and the memory manipulations are done b
 1. Forget Gate

The forget gate decides which information should be kept or removed from the

Values close to 0 remove information
 Values close to 1 retain information
 Helps discard unnecessary past information
 Controls memory retention in the LSTM

The equation for the forget gate is:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Where

W_f represents the weight matrix associated with the forget gate.
 $[h_{t-1}, x_t]$ denotes the concatenation of the current input and the prev
 b_f is the bias with the forget gate.
 σ is the sigmoid activation function.

forget_gate
 Forget Gate
 2. Input gate

The addition of useful information to the cell state is done by the input ga

First the information is regulated using the sigmoid function and filter
 Then, a vector is created using tanh function that gives an output from

At last the values of the vector and the regulated values are multiplied

The equation for the input gate is:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\hat{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

We multiply the previous state by f_t effectively filtering out the informat

$$C_t = f_t \odot C_{t-1} + i_t \odot \hat{C}_t$$

where

$\odot$ denotes element-wise multiplication

$\tanh$ is activation function

input_gate

Input Gate

3. Output gate

The output gate determines which information from the current cell state sho

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Next, the current cell state C_t is passed through a tanh activation to scal

$$h_t = o_t \odot \tanh(C_t)$$

Here:

o_t is the output gate activation.

C_t is the current cell state.

$\odot$ represents element-wise multiplication.

σ is the sigmoid activation function.

This hidden state h_t is then passed to the next time step and can also be

output_gate

Output Gate

Applications

Language modeling for machine translation and text summarization

Speech recognition for converting audio into text

Time series forecasting for stock prices, weather and energy usage

Anomaly detection for fraud and intrusion detection

Recommender systems for personalized suggestions

Video analysis for activity recognition and motion understanding

Understanding of LSTM Networks

Visit Coursecourse link icon

Suggested Quiz

4 Questions

What is the main purpose of using LSTM networks over traditional RNNs?

A

LSTMs require fewer computations during training

B

LSTMs retain information across longer sequences

C

LSTMs use a simpler network architecture

D

LSTMs are designed mainly for image analysis

success

Login to View Explanation

1/4

Comment

A

aakarshachug

Article Tags:

Deep Learning

Explore

Basics

Introduction

5 min read

AI vs ML vs DL

3 min read

Applications

3 min read

Challenges

7 min read

Importance

5 min read

Neural Networks

Introduction

9 min read

Types

5 min read

Layers in ANN

4 min read

Activation functions

6 min read

Feedforward Neural Network

6 min read

Backpropagation

7 min read

Deep Learning Models

CNN

5 min read

RNN

10 min read

- LSTM
 - 4 min read
- GRU
 - 5 min read
- Transformers
 - 5 min read
- Autoencoders
 - 6 min read
- GAN
 - 10 min read
- Model Evaluation
 - Gradient Descent
 - 13 min read
 - Momentum-based Gradient Optimizer
 - 3 min read
 - Adagrad
 - 6 min read
 - RMSProp
 - 4 min read
 - Adam Optimizer
 - 5 min read
- Deep Learning Frameworks
 - TensorFlow
 - 2 min read
 - PyTorch
 - 5 min read
 - Caffe
 - 8 min read
 - Apache MXNet
 - 4 min read
 - Theano
 - 4 min read
- Projects
 - Lung Cancer Detection
 - 4 min read
 - Cat & Dog Classification
 - 5 min read
 - Sentiment Analysis
 - 3 min read
 - Text Generation using LSTM
 - 4 min read
 - Machine Translation
 - 5 min read
 - Interview Questions
 - 15+ min read
- Courses
 - Data Science and ML Course
 - 2 min read
 - Data Analytics Course
 - 2 min read
 - Generative AI Course
 - 2 min read

GeeksforGeeks

location

Corporate & Communications Address:

A-143, 6th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh
location

Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddha Nagar

GFG App on Play Store

GFG App on App Store

Company

About Us

Legal

Privacy Policy

Contact Us

Advertise with us

GFG Corporate Solution

Campus Training Program

Explore

POTD

Job-A-Thon

Blogs

Nation Skill Up

Tutorials

Programming Languages

DSA

Web Technology

AI, ML & Data Science

DevOps

CS Core Subjects

Interview Preparation

Software and Tools

Courses

ML and Data Science

DSA and Placements

Web Development

Programming Languages

DevOps & Cloud

GATE

Trending Technologies

Videos

DSA

Python

Java

C++

Web Development

Data Science

CS Subjects

Preparation Corner

Interview Corner

Aptitude

Puzzles

GfG 160

System Design

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved ""

Why Do We Need Tokenization?

Deep learning models such as **LSTM** cannot process raw text directly. They can only work with **numerical values**. Therefore, we must convert text into numbers before feeding it to the model.

Example

Input sentence:

"My name is Srabon"

After Tokenization:

Word	Token ID
My	1
name	2
is	3
Srabon	4

Tokenized sequence:

[1, 2, 3, 4]

```
In [2]: import tensorflow as tf
        from tensorflow.keras.preprocessing.text import Tokenizer
```

```
WARNING: All log messages before absl::InitializeLog() is called are written
to STDERR
I0000 00:00:1783975209.635558 525085 cudart_stub.cc:31] Could not find cuda
drivers on your machine, GPU will not be used.
I0000 00:00:1783975209.770636 525085 cpu_feature_guard.cc:227] This TensorF
low binary is optimized to use available CPU instructions in performance-critical
operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
WARNING: All log messages before absl::InitializeLog() is called are written
to STDERR
I0000 00:00:1783975222.337308 525085 cudart_stub.cc:31] Could not find cuda
drivers on your machine, GPU will not be used.
```

```
In [3]: tokenizer=Tokenizer()
```

```
In [4]: tokenizer.fit_on_texts([text]) # Multiple texts can be passed in the form
```

```
In [5]: tokenizer.word_index
```

```
Out[5]: {'the': 1,
        'min': 2,
        'read': 3,
        't': 4,
        'to': 5,
        'and': 6,
        'gate': 7,
        'information': 8,
        'is': 9,
        'l': 10,
        'state': 11,
        'c': 12,
        'memory': 13,
        'lstm': 14,
        'a': 15,
        'for': 16,
        'of': 17,
        'h': 18,
        'cell': 19,
        'output': 20,
        'term': 21,
        '5': 22,
        'data': 23,
        'long': 24,
        'input': 25,
        'forget': 26,
        'x': 27,
        'values': 28,
        '4': 29,
        'hidden': 30,
        'which': 31,
        'in': 32,
        'time': 33,
        'by': 34,
        'using': 35,
        'from': 36,
        'model': 37,
        'current': 38,
        'function': 39,
        'f': 40,
        'lstms': 41,
        'with': 42,
        'be': 43,
        'b': 44,
        'activation': 45,
        'odot': 46,
        'o': 47,
        'science': 48,
        'network': 49,
        'dependencies': 50,
        'it': 51,
        'are': 52,
        'previous': 53,
        'learn': 54,
        'this': 55,
        'sigmoid': 56,
```

'sigma': 57,
'w': 58,
'2': 59,
'i': 60,
'3': 61,
't\u200b': 62,
'courses': 63,
'interview': 64,
'dsa': 65,
'learning': 66,
'introduction': 67,
'short': 68,
'rnn': 69,
'designed': 70,
'over': 71,
'gradient': 72,
'should': 73,
'passed': 74,
'then': 75,
'left': 76,
'cdot': 77,
'\right': 78,
'we': 79,
'ml': 80,
'6': 81,
'gfg': 82,
'geeksforgeeks': 83,
'machine': 84,
'devops': 85,
'neural': 86,
'sequential': 87,
'uses': 88,
'store': 89,
'rnns': 90,
'applications': 91,
'translation': 92,
'recognition': 93,
'that': 94,
'training': 95,
'can': 96,
'as': 97,
'through': 98,
'architecture': 99,
'cells': 100,
'gates': 101,
'next': 102,
'represents': 103,
'tanh': 104,
'element': 105,
'wise': 106,
'\u200b': 107,
'text': 108,
'detection': 109,
'analysis': 110,
'networks': 111,
'deep': 112,

'ai': 113,
'course': 114,
'corporate': 115,
'app': 116,
'us': 117,
'web': 118,
'tutorials': 119,
'java': 120,
'python': 121,
'last': 122,
'an': 123,
'limitations': 124,
'traditional': 125,
'longer': 126,
'sequences': 127,
'stores': 128,
'updates': 129,
'used': 130,
'language': 131,
'speech': 132,
'series': 133,
'forecasting': 134,
'steps': 135,
'they': 136,
'gradients': 137,
'makes': 138,
'earlier': 139,
'control': 140,
'flow': 141,
'main': 142,
'decides': 143,
'new': 144,
'determines': 145,
'removed': 146,
'controls': 147,
'done': 148,
'between': 149,
'0': 150,
'close': 151,
'retain': 152,
'equation': 153,
'where': 154,
'denotes': 155,
'useful': 156,
'regulated': 157,
'vector': 158,
'all': 159,
'at': 160,
'multiplied': 161,
'hat': 162,
'anh': 163,
'decided': 164,
'multiplication': 165,
'step': 166,
'understanding': 167,
'questions': 168,

'explore': 169,
'vs': 170,
'7': 171,
'10': 172,
'optimizer': 173,
'location': 174,
'address': 175,
'tower': 176,
'sector': 177,
'noida': 178,
'uttar': 179,
'pradesh': 180,
'201305': 181,
'k': 182,
'on': 183,
'programming': 184,
'languages': 185,
'cs': 186,
'subjects': 187,
'preparation': 188,
'development': 189,
'corner': 190,
'prep': 191,
'practice': 192,
'problems': 193,
'javascript': 194,
'linux': 195,
'updated': 196,
'18': 197,
'may': 198,
'2026': 199,
'improved': 200,
'version': 201,
'recurrent': 202,
'capture': 203,
'solving': 204,
'handles': 205,
'remembers': 206,
'important': 207,
'better': 208,
'than': 209,
'overcomes': 210,
'webpintroduction': 211,
'webp': 212,
'problem': 213,
'handle': 214,
'however': 215,
'struggle': 216,
'happens': 217,
'due': 218,
'vanishing': 219,
'when': 220,
'help': 221,
'shrink': 222,
'pass': 223,
'many': 224,

'hard': 225,
'patterns': 226,
'since': 227,
'becomes': 228,
'almost': 229,
'irrelevant': 230,
'exploding': 231,
'sometimes': 232,
'grow': 233,
'too': 234,
'large': 235,
'causing': 236,
'instability': 237,
'difficult': 238,
'properly': 239,
'become': 240,
'erratic': 241,
'unpredictable': 242,
'added': 243,
'working': 244,
'consists': 245,
'repeating': 246,
'chain': 247,
'like': 248,
'structure': 249,
'gating': 250,
'mechanisms': 251,
'retained': 252,
'manipulations': 253,
'thegates': 254,
'there': 255,
'three': 256,
'kept': 257,
'or': 258,
'applies': 259,
'generate': 260,
'remove': 261,
'helps': 262,
'discard': 263,
'unnecessary': 264,
'past': 265,
'retention': 266,
'weight': 267,
'matrix': 268,
'associated': 269,
'concatenation': 270,
'bias': 271,
'addition': 272,
'first': 273,
'filter': 274,
'remembered': 275,
'similar': 276,
'inputs': 277,
'created': 278,
'gives': 279,
'contains': 280,

'possible': 281,
'obtain': 282,
'multiply': 283,
'effectively': 284,
'filtering': 285,
'out': 286,
'had': 287,
'ignore': 288,
'add': 289,
'candidate': 290,
'scaled': 291,
'how': 292,
'much': 293,
'update': 294,
'each': 295,
'value': 296,
'followed': 297,
'scale': 298,
'its': 299,
'finally': 300,
'transformed': 301,
'produce': 302,
'here': 303,
'also': 304,
'generating': 305,
'modeling': 306,
'summarization': 307,
'converting': 308,
'audio': 309,
'into': 310,
'stock': 311,
'prices': 312,
'weather': 313,
'energy': 314,
'usage': 315,
'anomaly': 316,
'fraud': 317,
'intrusion': 318,
'recommender': 319,
'systems': 320,
'personalized': 321,
'suggestions': 322,
'video': 323,
'activity': 324,
'motion': 325,
'visit': 326,
'coursecourse': 327,
'link': 328,
'icon': 329,
'suggested': 330,
'quiz': 331,
'what': 332,
'purpose': 333,
'require': 334,
'fewer': 335,
'computations': 336,

'during': 337,
'across': 338,
'use': 339,
'simpler': 340,
'd': 341,
'mainly': 342,
'image': 343,
'success': 344,
'login': 345,
'view': 346,
'explanation': 347,
'comment': 348,
'aakarshachug': 349,
'article': 350,
'tags': 351,
'basics': 352,
'dl': 353,
'challenges': 354,
'importance': 355,
'9': 356,
'types': 357,
'layers': 358,
'ann': 359,
'functions': 360,
'feedforward': 361,
'backpropagation': 362,
'models': 363,
'cnn': 364,
'gru': 365,
'transformers': 366,
'autoencoders': 367,
'gan': 368,
'evaluation': 369,
'descent': 370,
'13': 371,
'momentum': 372,
'based': 373,
'adagrad': 374,
'rmsprop': 375,
'adam': 376,
'frameworks': 377,
'tensorflow': 378,
'pytorch': 379,
'caffe': 380,
'8': 381,
'apache': 382,
'mxnet': 383,
'theano': 384,
'projects': 385,
'lung': 386,
'cancer': 387,
'cat': 388,
'dog': 389,
'classification': 390,
'sentiment': 391,
'generation': 392,

```
'15': 393,  
'analytics': 394,  
'generative': 395,  
'communications': 396,  
'143': 397,  
'6th': 398,  
'floor': 399,  
'sovereign': 400,  
'136': 401,  
'registered': 402,  
'061': 403,  
'gulshan': 404,  
'vivante': 405,  
'apartment': 406,  
'137': 407,  
'gautam': 408,  
'buddh': 409,  
'nagar': 410,  
'play': 411,  
'company': 412,  
'about': 413,  
'legal': 414,  
'privacy': 415,  
'policy': 416,  
'contact': 417,  
'advertise': 418,  
'solution': 419,  
'campus': 420,  
'program': 421,  
'potd': 422,  
'job': 423,  
'thon': 424,  
'blogs': 425,  
'nation': 426,  
'skill': 427,  
'up': 428,  
'technology': 429,  
'core': 430,  
'software': 431,  
'tools': 432,  
'placements': 433,  
'cloud': 434,  
'trending': 435,  
'technologies': 436,  
'videos': 437,  
'aptitude': 438,  
'puzzles': 439,  
'160': 440,  
'system': 441,  
'design': 442,  
'sanchhaya': 443,  
'education': 444,  
'private': 445,  
'limited': 446,  
'rights': 447,  
'reserved': 448}
```

```
In [6]: print(len(tokenizer.word_index)) # Printing the total number of unique words
```

448

Above we see how the text assigned to numbers

- Now Create dataset like supervised learnign:
- Let see a small example:

"My name is Srabon"

After Tokenization:

Word	Token ID
My	1
name	2
is	3
Srabon	4

Dataset Created

Input	Output
My	name
My name	is
My name is	Srabon

Tokenized Dataset

Input	Output
[1]	[2]
[1,2]	[3]
[1,2,3]	[4]

```
In [7]: for sentence in text.split("\n"):
        print(sentence) # Paragraph wise printing of the text. Each paragraph is
```

geeksforgeeks

Courses
Tutorials
Interview Prep

DSA
Practice Problems
C
C++
Java
Python
JavaScript
Data Science
Machine Learning
Courses
Linux
DevOps

Introduction to Long Short Term Memory
Last Updated : 18 May, 2026

Long Short-Term Memory (LSTM) is an improved version of the Recurrent Neural Network (RNN) designed to capture long-term dependencies in sequential data. It uses a memory cell to store information over time, solving the limitations of traditional RNNs.

Handles Long Term Dependencies: Remembers information for longer sequences
Memory Cell: Stores and updates important information over time
Better than RNN: Overcomes short term memory limitations
Applications: Used in language translation, speech recognition and time series forecasting

introduction_to_lstm.webpintroduction_to_lstm.webp
Problem with Long-Term Dependencies in RNN

RNNs are designed to handle sequential data by using a hidden state that stores information from previous steps. However, they struggle to learn long-term dependencies. This happens due to:

Vanishing Gradient: When training a model over time, the gradients which help the model learn can shrink as they pass through many steps. This makes it hard for the model to learn long-term patterns since earlier information becomes almost irrelevant.

Exploding Gradient: Sometimes gradients can grow too large causing instability. This makes it difficult for the model to learn properly as the updates to the model become erratic and unpredictable.

LSTM Architecture

LSTM (Long Short-Term Memory) architecture is designed to learn long-term dependencies in sequential data using memory cells and gates that control the flow of information through the network.

Main Gates in LSTM

Input Gate: Decides which new information should be added to the memory cell

Forget Gate: Determines which information should be removed from the memory cell

Output Gate: Controls which information from the memory cell is passed to the next hidden state and output

Working of LSTM

LSTM consists of a repeating chain like structure with memory cells and gating mechanisms

gate_of_lstm

LSTM Model

Information is retained by the cells and the memory manipulations are done by the gates. There are three gates:

1. Forget Gate

The forget gate decides which information should be kept or removed from the cell state. It uses the current input x_t and previous hidden state h_{t-1} then applies a sigmoid function to generate values between 0 and 1.

Values close to 0 remove information

Values close to 1 retain information

Helps discard unnecessary past information

Controls memory retention in the LSTM

The equation for the forget gate is:

$$\text{forget_gate}_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Where

W_f represents the weight matrix associated with the forget gate.

$[h_{t-1}, x_t]$ denotes the concatenation of the current input and the previous hidden state.

b_f is the bias with the forget gate.

σ is the sigmoid activation function.

forget_gate

Forget Gate

2. Input gate

The addition of useful information to the cell state is done by the input gate.

First the information is regulated using the sigmoid function and filter the values to be remembered similar to the forget gate using inputs h_{t-1} and x_t .

Then, a vector is created using tanh function that gives an output from -1 to +1 which contains all the possible values from h_{t-1} and x_t .

At last the values of the vector and the regulated values are multiplied to obtain the useful information.

The equation for the input gate is:

$$\text{ight)}_t = \sigma \left(W_i \cdot [h_{t-1}, x_t] + b_i \right)$$

$$\text{ight)}_{\hat{C}}_t = \tanh \left(W_c \cdot [h_{t-1}, x_t] + b_c \right)$$

We multiply the previous state by f_t effectively filtering out the information we had decided to ignore earlier. Then we add $i_t \odot C_t$ which represents the new candidate values scaled by how much we decided to update each state value.

$$C_t = f_t \odot C_{t-1} + i_t \odot \hat{C}_t$$

where

$\odot$ denotes element-wise multiplication
 $\tanh$ is activation function

input_gate

Input Gate

3. Output gate

The output gate determines which information from the current cell state should be passed as the hidden state (output) at the current time step. It uses the previous hidden state h_{t-1} and the current input x_t , followed by a sigmoid function to control the output flow.

$$\text{ight)}_t = \sigma \left(W_o \cdot [h_{t-1}, x_t] + b_o \right)$$

Next, the current cell state C_t is passed through a $\tanh$ activation to scale its values between -1 and +1. Finally, this transformed cell state is multiplied element-wise with o_t to produce the hidden state h_t :

$$h_t = o_t \odot \tanh(C_t)$$

Here:

o_t is the output gate activation.

C_t is the current cell state.

$\odot$ represents element-wise multiplication.

σ is the sigmoid activation function.

This hidden state h_t is then passed to the next time step and can also be used for generating the output of the network.

output_gate

Output Gate

Applications

Language modeling for machine translation and text summarization

Speech recognition for converting audio into text

Time series forecasting for stock prices, weather and energy usage

Anomaly detection for fraud and intrusion detection

Recommender systems for personalized suggestions

Video analysis for activity recognition and motion understanding

Understanding of LSTM Networks

Visit Coursecourse link icon

Suggested Quiz

4 Questions

What is the main purpose of using LSTM networks over traditional RNNs?

A

LSTMs require fewer computations during training

B

LSTMs retain information across longer sequences

C

LSTMs use a simpler network architecture

D

LSTMs are designed mainly for image analysis

success

Login to View Explanation

1/4

Comment

A

aakarshachug

Article Tags:

Deep Learning

Explore

Basics

Introduction

5 min read

AI vs ML vs DL

3 min read

Applications

3 min read

Challenges

7 min read

Importance

5 min read

Neural Networks

Introduction

9 min read

Types

5 min read

Layers in ANN

4 min read

Activation functions

6 min read

Feedforward Neural Network

6 min read

Backpropagation

7 min read

Deep Learning Models

CNN

5 min read

RNN

10 min read

LSTM

4 min read

GRU

5 min read

Transformers

5 min read

Autoencoders

6 min read

GAN

10 min read

Model Evaluation

Gradient Descent

13 min read

Momentum-based Gradient Optimizer

3 min read

Adagrad

6 min read

RMSProp

4 min read

Adam Optimizer

5 min read

Deep Learning Frameworks

TensorFlow

2 min read

PyTorch

5 min read

Caffe

8 min read

Apache MXNet

4 min read

Theano

4 min read

Projects

Lung Cancer Detection

4 min read

Cat & Dog Classification

5 min read

Sentiment Analysis

3 min read

Text Generation using LSTM

4 min read

Machine Translation

5 min read

Interview Questions

15+ min read

Courses

Data Science and ML Course

2 min read

Data Analytics Course

2 min read

Generative AI Course

2 min read

GeeksforGeeks

location

Corporate & Communications Address:

A-143, 6th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

location

Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddha Nagar, Uttar Pradesh, 201305

GFG App on Play Store

GFG App on App Store

Company

About Us

Legal

Privacy Policy

Contact Us

Advertise with us

GFG Corporate Solution

Campus Training Program

Explore

POTD

Job-A-Thon

Blogs

Nation Skill Up

Tutorials

Programming Languages

DSA

Web Technology

AI, ML & Data Science

DevOps

CS Core Subjects

Interview Preparation

Software and Tools

Courses

ML and Data Science

DSA and Placements

Web Development

Programming Languages

DevOps & Cloud

GATE

Trending Technologies

Videos

DSA

Python

Java

C++

Web Development

Data Science

CS Subjects

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved

```
In [8]: for sentence in text.split("\n"):
        print(tokenizer.texts_to_sequences([sentence])[0]) # Paragraph wise pri
```

```
[83]
[]
[63]
[119]
[64, 191]
[]
[65]
[192, 193]
[12]
[12]
[120]
[121]
[194]
[23, 48]
[84, 66]
[63]
[195]
[85]
[]
[67, 5, 24, 68, 21, 13]
[122, 196, 197, 198, 199]
[]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51, 88, 15, 13, 19, 5, 89, 8, 71, 33, 204, 1, 124, 1
7, 125, 90]
[]
[205, 24, 21, 50, 206, 8, 16, 126, 127]
[13, 19, 128, 6, 129, 207, 8, 71, 33]
[208, 209, 69, 210, 68, 21, 13, 124]
[91, 130, 32, 131, 92, 132, 93, 6, 33, 133, 134]
[]
[67, 5, 41, 211, 5, 41, 212]
[213, 42, 24, 21, 50, 32, 69]
[]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8, 36, 53, 135, 21
5, 136, 216, 5, 54, 24, 21, 50, 55, 217, 218, 5]
[]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97,
136, 223, 98, 224, 135, 55, 138, 51, 225, 16, 1, 37, 5, 54, 24, 21, 226, 22
7, 139, 8, 228, 229, 230]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16, 1, 3
7, 5, 54, 239, 97, 1, 129, 5, 1, 37, 240, 241, 6, 242]
[]
[14, 99]
[]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87, 23, 35, 13, 100,
6, 101, 94, 140, 1, 141, 17, 8, 98, 1, 49]
[142, 101, 32, 14]
[]
[25, 7, 143, 31, 144, 8, 73, 43, 243, 5, 1, 13, 19]
[26, 7, 145, 31, 8, 73, 43, 146, 36, 1, 13, 19]
[20, 7, 147, 31, 8, 36, 1, 13, 19, 9, 74, 5, 1, 102, 30, 11, 6, 20]
[]
[244, 17, 14]
[]
[14, 245, 17, 15, 246, 247, 248, 249, 42, 13, 100, 6, 250, 251]
```

```
[7, 17, 14]
[14, 37]
[]
[8, 9, 252, 34, 1, 100, 6, 1, 13, 253, 52, 148, 34, 254, 255, 52, 256, 101]
[10, 26, 7]
[]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38,
25, 27, 4, 6, 53, 30, 11, 18, 4, 10, 75, 259, 15, 56, 39, 5, 260, 28, 149, 1
50, 6, 10]
[]
[28, 151, 5, 150, 261, 8]
[28, 151, 5, 10, 152, 8]
[262, 263, 264, 265, 8]
[147, 13, 266, 32, 1, 14]
[]
[1, 153, 16, 1, 26, 7, 9]
[]
[40, 4, 57, 76, 58, 40, 77, 18, 4, 10, 27, 4, 44, 40, 78]
[]
[154]
[]
[58, 40, 103, 1, 267, 268, 269, 42, 1, 26, 7]
[18, 4, 10, 27, 4, 155, 1, 270, 17, 1, 38, 25, 6, 1, 53, 30, 11]
[44, 40, 9, 1, 271, 42, 1, 26, 7]
[57, 9, 1, 56, 45, 39]
[]
[26, 7]
[26, 7]
[59, 25, 7]
[]
[1, 272, 17, 156, 8, 5, 1, 19, 11, 9, 148, 34, 1, 25, 7]
[]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5, 43, 275, 276, 5, 1, 26,
7, 35, 277, 18, 4, 10, 6, 27, 4]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10, 5, 10, 31, 280,
159, 1, 281, 28, 36, 18, 4, 10, 6, 27, 4]
[160, 122, 1, 28, 17, 1, 158, 6, 1, 157, 28, 52, 161, 5, 282, 1, 156, 8]
[]
[1, 153, 16, 1, 25, 7, 9]
[]
[60, 4, 57, 76, 58, 60, 77, 18, 4, 10, 27, 4, 44, 60, 78]
[]
[162, 12, 4, 163, 76, 58, 12, 77, 18, 4, 10, 27, 4, 44, 12, 78]
[]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103, 1, 144, 290, 28, 291, 34, 292, 2
93, 79, 164, 5, 294, 295, 11, 296]
[]
[12, 4, 40, 4, 46, 12, 4, 10, 60, 4, 46, 162, 12, 4]
[]
[154]
[]
[46, 155, 105, 106, 165]
[104, 9, 45, 39]
[]
[25, 7]
```

```
[25, 7]
[61, 20, 7]
[]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1, 38, 25, 27,
62, 107, 297, 34, 15, 56, 39, 5, 140, 1, 20, 141]
[]
[47, 4, 57, 76, 58, 47, 77, 18, 4, 10, 27, 4, 44, 47, 78]
[]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19, 11, 9, 161, 105, 106, 42, 47, 62, 5, 302, 1, 30,
11, 18, 4]
[]
[18, 4, 47, 4, 46, 163, 12, 4]
[]
[303]
[]
[47, 62, 9, 1, 20, 7, 45]
[12, 62, 9, 1, 38, 19, 11]
[46, 103, 105, 106, 165]
[57, 9, 1, 56, 45, 39]
[]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1, 102, 33, 166, 6, 96, 304, 43, 130,
16, 305, 1, 20, 17, 1, 49]
[20, 7]
[20, 7]
[91]
[]
[131, 306, 16, 84, 92, 6, 108, 307]
[132, 93, 16, 308, 309, 310, 108]
[33, 133, 134, 16, 311, 312, 313, 6, 314, 315]
[316, 109, 16, 317, 6, 318, 109]
[319, 320, 16, 321, 322]
[323, 110, 16, 324, 93, 6, 325, 167]
[]
[167, 17, 14, 111]
[326, 327, 328, 329]
[330, 331]
[]
[29, 168]
[]
[332, 9, 1, 142, 333, 17, 35, 14, 111, 71, 125, 90]
[]
[15]
[]
[41, 334, 335, 336, 337, 95]
[44]
[]
[41, 152, 8, 338, 126, 127]
[12]
[]
[41, 339, 15, 340, 49, 99]
[341]
[]
[41, 52, 70, 342, 16, 343, 110]
[]
```

```
[ ]
[344]
[ ]
[345, 5, 346, 347]
[ ]
[10, 29]
[348]
[ ]
[15]
[349]
[350, 351]
[112, 66]
[169]
[ ]
[352]
[67]
[22, 2, 3]
[113, 170, 80, 170, 353]
[61, 2, 3]
[91]
[61, 2, 3]
[354]
[171, 2, 3]
[355]
[22, 2, 3]
[86, 111]
[67]
[356, 2, 3]
[357]
[22, 2, 3]
[358, 32, 359]
[29, 2, 3]
[45, 360]
[81, 2, 3]
[361, 86, 49]
[81, 2, 3]
[362]
[171, 2, 3]
[112, 66, 363]
[364]
[22, 2, 3]
[69]
[172, 2, 3]
[14]
[29, 2, 3]
[365]
[22, 2, 3]
[366]
[22, 2, 3]
[367]
[81, 2, 3]
[368]
[172, 2, 3]
[37, 369]
[72, 370]
[371, 2, 3]
```

[372, 373, 72, 173]
[61, 2, 3]
[374]
[81, 2, 3]
[375]
[29, 2, 3]
[376, 173]
[22, 2, 3]
[112, 66, 377]
[378]
[59, 2, 3]
[379]
[22, 2, 3]
[380]
[381, 2, 3]
[382, 383]
[29, 2, 3]
[384]
[29, 2, 3]
[385]
[386, 387, 109]
[29, 2, 3]
[388, 389, 390]
[22, 2, 3]
[391, 110]
[61, 2, 3]
[108, 392, 35, 14]
[29, 2, 3]
[84, 92]
[22, 2, 3]
[64, 168]
[393, 2, 3]
[63]
[23, 48, 6, 80, 114]
[59, 2, 3]
[23, 394, 114]
[59, 2, 3]
[395, 113, 114]
[59, 2, 3]
[]
[83]
[174]
[115, 396, 175]
[15, 397, 398, 399, 400, 115, 176, 177, 401, 178, 179, 180, 181]
[174]
[402, 175]
[182, 403, 176, 182, 404, 405, 406, 177, 407, 178, 408, 409, 410, 179, 180, 181]
[82, 116, 183, 411, 89]
[82, 116, 183, 116, 89]
[]
[412]
[413, 117]
[414]
[415, 416]
[417, 117]


```
[418, 42, 117]
[82, 115, 419]
[420, 95, 421]
[]
[169]
[422]
[423, 15, 424]
[425]
[426, 427, 428]
[]
[119]
[184, 185]
[65]
[118, 429]
[113, 80, 23, 48]
[85]
[186, 430, 187]
[64, 188]
[431, 6, 432]
[]
[63]
[80, 6, 23, 48]
[65, 6, 433]
[118, 189]
[184, 185]
[85, 434]
[7]
[435, 436]
[]
[437]
[65]
[121]
[120]
[12]
[118, 189]
[23, 48]
[186, 187]
[]
[188, 190]
[64, 190]
[438]
[439]
[82, 440]
[441, 442]
[]
[83, 443, 444, 445, 446, 159, 447, 448]
```

Now we make out dataset use those vectors using loop :

```
In [9]: input_sequences = []
        for sentence in text.split("\n"):
            token_vector=tokenizer.texts_to_sequences([sentence])[0]
            for i in range(1, len(token_vector)):
                n_gram_sequence=token_vector[:i+1]
                input_sequences.append(n_gram_sequence) # Adding the n-gram sequence

In [10]: for i in input_sequences:
           print(i) # Printing the n-gram sequences generated from the text. Each
```

[64, 191]
[192, 193]
[23, 48]
[84, 66]
[67, 5]
[67, 5, 24]
[67, 5, 24, 68]
[67, 5, 24, 68, 21]
[67, 5, 24, 68, 21, 13]
[122, 196]
[122, 196, 197]
[122, 196, 197, 198]
[122, 196, 197, 198, 199]
[24, 68]
[24, 68, 21]
[24, 68, 21, 13]
[24, 68, 21, 13, 14]
[24, 68, 21, 13, 14, 9]
[24, 68, 21, 13, 14, 9, 123]
[24, 68, 21, 13, 14, 9, 123, 200]
[24, 68, 21, 13, 14, 9, 123, 200, 201]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51, 88]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51, 88, 15]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51, 88, 15, 13]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51, 88, 15, 13, 19]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51, 88, 15, 13, 19, 5]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51, 88, 15, 13, 19, 5, 89]

[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51, 88, 15, 13, 19, 5, 89, 8]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51, 88, 15, 13, 19, 5, 89, 8, 71]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51, 88, 15, 13, 19, 5, 89, 8, 71, 33]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51, 88, 15, 13, 19, 5, 89, 8, 71, 33, 204]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51, 88, 15, 13, 19, 5, 89, 8, 71, 33, 204, 1]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51, 88, 15, 13, 19, 5, 89, 8, 71, 33, 204, 1, 124]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51, 88, 15, 13, 19, 5, 89, 8, 71, 33, 204, 1, 124, 1
7]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51, 88, 15, 13, 19, 5, 89, 8, 71, 33, 204, 1, 124, 1
7, 125]
[24, 68, 21, 13, 14, 9, 123, 200, 201, 17, 1, 202, 86, 49, 69, 70, 5, 203, 2
4, 21, 50, 32, 87, 23, 51, 88, 15, 13, 19, 5, 89, 8, 71, 33, 204, 1, 124, 1
7, 125, 90]
[205, 24]
[205, 24, 21]
[205, 24, 21, 50]
[205, 24, 21, 50, 206]
[205, 24, 21, 50, 206, 8]
[205, 24, 21, 50, 206, 8, 16]
[205, 24, 21, 50, 206, 8, 16, 126]
[205, 24, 21, 50, 206, 8, 16, 126, 127]
[13, 19]
[13, 19, 128]
[13, 19, 128, 6]
[13, 19, 128, 6, 129]
[13, 19, 128, 6, 129, 207]
[13, 19, 128, 6, 129, 207, 8]
[13, 19, 128, 6, 129, 207, 8, 71]
[13, 19, 128, 6, 129, 207, 8, 71, 33]
[208, 209]
[208, 209, 69]
[208, 209, 69, 210]
[208, 209, 69, 210, 68]
[208, 209, 69, 210, 68, 21]
[208, 209, 69, 210, 68, 21, 13]
[208, 209, 69, 210, 68, 21, 13, 124]
[91, 130]
[91, 130, 32]
[91, 130, 32, 131]
[91, 130, 32, 131, 92]
[91, 130, 32, 131, 92, 132]
[91, 130, 32, 131, 92, 132, 93]
[91, 130, 32, 131, 92, 132, 93, 6]
[91, 130, 32, 131, 92, 132, 93, 6, 33]
[91, 130, 32, 131, 92, 132, 93, 6, 33, 133]
[91, 130, 32, 131, 92, 132, 93, 6, 33, 133, 134]
[67, 5]
[67, 5, 41]

[67, 5, 41, 211]
[67, 5, 41, 211, 5]
[67, 5, 41, 211, 5, 41]
[67, 5, 41, 211, 5, 41, 212]
[213, 42]
[213, 42, 24]
[213, 42, 24, 21]
[213, 42, 24, 21, 50]
[213, 42, 24, 21, 50, 32]
[213, 42, 24, 21, 50, 32, 69]
[90, 52]
[90, 52, 70]
[90, 52, 70, 5]
[90, 52, 70, 5, 214]
[90, 52, 70, 5, 214, 87]
[90, 52, 70, 5, 214, 87, 23]
[90, 52, 70, 5, 214, 87, 23, 34]
[90, 52, 70, 5, 214, 87, 23, 34, 35]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8, 36]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8, 36, 53]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8, 36, 53, 135]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8, 36, 53, 135, 21
5]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8, 36, 53, 135, 21
5, 136]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8, 36, 53, 135, 21
5, 136, 216]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8, 36, 53, 135, 21
5, 136, 216, 5]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8, 36, 53, 135, 21
5, 136, 216, 5, 54]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8, 36, 53, 135, 21
5, 136, 216, 5, 54, 24]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8, 36, 53, 135, 21
5, 136, 216, 5, 54, 24, 21]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8, 36, 53, 135, 21
5, 136, 216, 5, 54, 24, 21, 50]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8, 36, 53, 135, 21
5, 136, 216, 5, 54, 24, 21, 50, 55]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8, 36, 53, 135, 21
5, 136, 216, 5, 54, 24, 21, 50, 55, 217]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8, 36, 53, 135, 21
5, 136, 216, 5, 54, 24, 21, 50, 55, 217, 218]
[90, 52, 70, 5, 214, 87, 23, 34, 35, 15, 30, 11, 94, 128, 8, 36, 53, 135, 21
5, 136, 216, 5, 54, 24, 21, 50, 55, 217, 218, 5]
[219, 72]
[219, 72, 220]
[219, 72, 220, 95]
[219, 72, 220, 95, 15]
[219, 72, 220, 95, 15, 37]

[219, 72, 220, 95, 15, 37, 71]
[219, 72, 220, 95, 15, 37, 71, 33]
[219, 72, 220, 95, 15, 37, 71, 33, 1]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224, 135]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224, 135, 55]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224, 135, 55, 138]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224, 135, 55, 138, 51]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224, 135, 55, 138, 51, 225]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224, 135, 55, 138, 51, 225, 16]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224, 135, 55, 138, 51, 225, 16, 1]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224, 135, 55, 138, 51, 225, 16, 1, 37]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224, 135, 55, 138, 51, 225, 16, 1, 37, 5]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224, 135, 55, 138, 51, 225, 16, 1, 37, 5, 54]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224, 135, 55, 138, 51, 225, 16, 1, 37, 5, 54, 24]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224, 135, 55, 138, 51, 225, 16, 1, 37, 5, 54, 24, 21]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224, 135, 55, 138, 51, 225, 16, 1, 37, 5, 54, 24, 21, 226]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224, 135, 55, 138, 51, 225, 16, 1, 37, 5, 54, 24, 21, 226, 22 7]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224, 135, 55, 138, 51, 225, 16, 1, 37, 5, 54, 24, 21, 226, 22 7, 139]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97, 136, 223, 98, 224, 135, 55, 138, 51, 225, 16, 1, 37, 5, 54, 24, 21, 226, 22 7, 139, 8]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97,

136, 223, 98, 224, 135, 55, 138, 51, 225, 16, 1, 37, 5, 54, 24, 21, 226, 22
7, 139, 8, 228]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97,
136, 223, 98, 224, 135, 55, 138, 51, 225, 16, 1, 37, 5, 54, 24, 21, 226, 22
7, 139, 8, 228, 229]
[219, 72, 220, 95, 15, 37, 71, 33, 1, 137, 31, 221, 1, 37, 54, 96, 222, 97,
136, 223, 98, 224, 135, 55, 138, 51, 225, 16, 1, 37, 5, 54, 24, 21, 226, 22
7, 139, 8, 228, 229, 230]
[231, 72]
[231, 72, 232]
[231, 72, 232, 137]
[231, 72, 232, 137, 96]
[231, 72, 232, 137, 96, 233]
[231, 72, 232, 137, 96, 233, 234]
[231, 72, 232, 137, 96, 233, 234, 235]
[231, 72, 232, 137, 96, 233, 234, 235, 236]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16, 1]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16, 1, 3
7]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16, 1, 3
7, 5]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16, 1, 3
7, 5, 54]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16, 1, 3
7, 5, 54, 239]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16, 1, 3
7, 5, 54, 239, 97]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16, 1, 3
7, 5, 54, 239, 97, 1]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16, 1, 3
7, 5, 54, 239, 97, 1, 129]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16, 1, 3
7, 5, 54, 239, 97, 1, 129, 5]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16, 1, 3
7, 5, 54, 239, 97, 1, 129, 5, 1]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16, 1, 3
7, 5, 54, 239, 97, 1, 129, 5, 1, 37]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16, 1, 3
7, 5, 54, 239, 97, 1, 129, 5, 1, 37, 240]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16, 1, 3
7, 5, 54, 239, 97, 1, 129, 5, 1, 37, 240, 241]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16, 1, 3
7, 5, 54, 239, 97, 1, 129, 5, 1, 37, 240, 241, 6]
[231, 72, 232, 137, 96, 233, 234, 235, 236, 237, 55, 138, 51, 238, 16, 1, 3
7, 5, 54, 239, 97, 1, 129, 5, 1, 37, 240, 241, 6, 242]
[14, 99]
[14, 24]
[14, 24, 68]
[14, 24, 68, 21]
[14, 24, 68, 21, 13]

[14, 24, 68, 21, 13, 99]
[14, 24, 68, 21, 13, 99, 9]
[14, 24, 68, 21, 13, 99, 9, 70]
[14, 24, 68, 21, 13, 99, 9, 70, 5]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87, 23]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87, 23, 35]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87, 23, 35, 13]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87, 23, 35, 13, 100]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87, 23, 35, 13, 100, 6]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87, 23, 35, 13, 100, 6, 101]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87, 23, 35, 13, 100, 6, 101, 94]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87, 23, 35, 13, 100, 6, 101, 94, 140]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87, 23, 35, 13, 100, 6, 101, 94, 140, 1]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87, 23, 35, 13, 100, 6, 101, 94, 140, 1, 141]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87, 23, 35, 13, 100, 6, 101, 94, 140, 1, 141, 17]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87, 23, 35, 13, 100, 6, 101, 94, 140, 1, 141, 17, 8]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87, 23, 35, 13, 100, 6, 101, 94, 140, 1, 141, 17, 8, 98]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87, 23, 35, 13, 100, 6, 101, 94, 140, 1, 141, 17, 8, 98, 1]
[14, 24, 68, 21, 13, 99, 9, 70, 5, 54, 24, 21, 50, 32, 87, 23, 35, 13, 100, 6, 101, 94, 140, 1, 141, 17, 8, 98, 1, 49]
[142, 101]
[142, 101, 32]
[142, 101, 32, 14]
[25, 7]
[25, 7, 143]
[25, 7, 143, 31]
[25, 7, 143, 31, 144]
[25, 7, 143, 31, 144, 8]
[25, 7, 143, 31, 144, 8, 73]
[25, 7, 143, 31, 144, 8, 73, 43]
[25, 7, 143, 31, 144, 8, 73, 43, 243]
[25, 7, 143, 31, 144, 8, 73, 43, 243, 5]
[25, 7, 143, 31, 144, 8, 73, 43, 243, 5, 1]
[25, 7, 143, 31, 144, 8, 73, 43, 243, 5, 1, 13]
[25, 7, 143, 31, 144, 8, 73, 43, 243, 5, 1, 13, 19]
[26, 7]
[26, 7, 145]
[26, 7, 145, 31]
[26, 7, 145, 31, 8]
[26, 7, 145, 31, 8, 73]

[26, 7, 145, 31, 8, 73, 43]
[26, 7, 145, 31, 8, 73, 43, 146]
[26, 7, 145, 31, 8, 73, 43, 146, 36]
[26, 7, 145, 31, 8, 73, 43, 146, 36, 1]
[26, 7, 145, 31, 8, 73, 43, 146, 36, 1, 13]
[26, 7, 145, 31, 8, 73, 43, 146, 36, 1, 13, 19]
[20, 7]
[20, 7, 147]
[20, 7, 147, 31]
[20, 7, 147, 31, 8]
[20, 7, 147, 31, 8, 36]
[20, 7, 147, 31, 8, 36, 1]
[20, 7, 147, 31, 8, 36, 1, 13]
[20, 7, 147, 31, 8, 36, 1, 13, 19]
[20, 7, 147, 31, 8, 36, 1, 13, 19, 9]
[20, 7, 147, 31, 8, 36, 1, 13, 19, 9, 74]
[20, 7, 147, 31, 8, 36, 1, 13, 19, 9, 74, 5]
[20, 7, 147, 31, 8, 36, 1, 13, 19, 9, 74, 5, 1]
[20, 7, 147, 31, 8, 36, 1, 13, 19, 9, 74, 5, 1, 102]
[20, 7, 147, 31, 8, 36, 1, 13, 19, 9, 74, 5, 1, 102, 30]
[20, 7, 147, 31, 8, 36, 1, 13, 19, 9, 74, 5, 1, 102, 30, 11]
[20, 7, 147, 31, 8, 36, 1, 13, 19, 9, 74, 5, 1, 102, 30, 11, 6]
[20, 7, 147, 31, 8, 36, 1, 13, 19, 9, 74, 5, 1, 102, 30, 11, 6, 20]
[244, 17]
[244, 17, 14]
[14, 245]
[14, 245, 17]
[14, 245, 17, 15]
[14, 245, 17, 15, 246]
[14, 245, 17, 15, 246, 247]
[14, 245, 17, 15, 246, 247, 248]
[14, 245, 17, 15, 246, 247, 248, 249]
[14, 245, 17, 15, 246, 247, 248, 249, 42]
[14, 245, 17, 15, 246, 247, 248, 249, 42, 13]
[14, 245, 17, 15, 246, 247, 248, 249, 42, 13, 100]
[14, 245, 17, 15, 246, 247, 248, 249, 42, 13, 100, 6]
[14, 245, 17, 15, 246, 247, 248, 249, 42, 13, 100, 6, 250]
[14, 245, 17, 15, 246, 247, 248, 249, 42, 13, 100, 6, 250, 251]
[7, 17]
[7, 17, 14]
[14, 37]
[8, 9]
[8, 9, 252]
[8, 9, 252, 34]
[8, 9, 252, 34, 1]
[8, 9, 252, 34, 1, 100]
[8, 9, 252, 34, 1, 100, 6]
[8, 9, 252, 34, 1, 100, 6, 1]
[8, 9, 252, 34, 1, 100, 6, 1, 13]
[8, 9, 252, 34, 1, 100, 6, 1, 13, 253]
[8, 9, 252, 34, 1, 100, 6, 1, 13, 253, 52]
[8, 9, 252, 34, 1, 100, 6, 1, 13, 253, 52, 148]
[8, 9, 252, 34, 1, 100, 6, 1, 13, 253, 52, 148, 34]
[8, 9, 252, 34, 1, 100, 6, 1, 13, 253, 52, 148, 34, 254]
[8, 9, 252, 34, 1, 100, 6, 1, 13, 253, 52, 148, 34, 254, 255]
[8, 9, 252, 34, 1, 100, 6, 1, 13, 253, 52, 148, 34, 254, 255, 52]

[8, 9, 252, 34, 1, 100, 6, 1, 13, 253, 52, 148, 34, 254, 255, 52, 256]
[8, 9, 252, 34, 1, 100, 6, 1, 13, 253, 52, 148, 34, 254, 255, 52, 256, 101]
[10, 26]
[10, 26, 7]
[1, 26]
[1, 26, 7]
[1, 26, 7, 143]
[1, 26, 7, 143, 31]
[1, 26, 7, 143, 31, 8]
[1, 26, 7, 143, 31, 8, 73]
[1, 26, 7, 143, 31, 8, 73, 43]
[1, 26, 7, 143, 31, 8, 73, 43, 257]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30, 11]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30, 11, 18]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30, 11, 18, 4]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30, 11, 18, 4, 10]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30, 11, 18, 4, 10, 75]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30, 11, 18, 4, 10, 75, 259]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30, 11, 18, 4, 10, 75, 259, 15]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30, 11, 18, 4, 10, 75, 259, 15, 56]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30, 11, 18, 4, 10, 75, 259, 15, 56, 39]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30, 11, 18, 4, 10, 75, 259, 15, 56, 39, 5]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30, 11, 18, 4, 10, 75, 259, 15, 56, 39, 5, 260]

[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30, 11, 18, 4, 10, 75, 259, 15, 56, 39, 5, 260, 28]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30, 11, 18, 4, 10, 75, 259, 15, 56, 39, 5, 260, 28, 149]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30, 11, 18, 4, 10, 75, 259, 15, 56, 39, 5, 260, 28, 149, 1 50]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30, 11, 18, 4, 10, 75, 259, 15, 56, 39, 5, 260, 28, 149, 1 50, 6]
[1, 26, 7, 143, 31, 8, 73, 43, 257, 258, 146, 36, 1, 19, 11, 51, 88, 1, 38, 25, 27, 4, 6, 53, 30, 11, 18, 4, 10, 75, 259, 15, 56, 39, 5, 260, 28, 149, 1 50, 6, 10]
[28, 151]
[28, 151, 5]
[28, 151, 5, 150]
[28, 151, 5, 150, 261]
[28, 151, 5, 150, 261, 8]
[28, 151]
[28, 151, 5]
[28, 151, 5, 10]
[28, 151, 5, 10, 152]
[28, 151, 5, 10, 152, 8]
[262, 263]
[262, 263, 264]
[262, 263, 264, 265]
[262, 263, 264, 265, 8]
[147, 13]
[147, 13, 266]
[147, 13, 266, 32]
[147, 13, 266, 32, 1]
[147, 13, 266, 32, 1, 14]
[1, 153]
[1, 153, 16]
[1, 153, 16, 1]
[1, 153, 16, 1, 26]
[1, 153, 16, 1, 26, 7]
[1, 153, 16, 1, 26, 7, 9]
[40, 4]
[40, 4, 57]
[40, 4, 57, 76]
[40, 4, 57, 76, 58]
[40, 4, 57, 76, 58, 40]
[40, 4, 57, 76, 58, 40, 77]
[40, 4, 57, 76, 58, 40, 77, 18]
[40, 4, 57, 76, 58, 40, 77, 18, 4]
[40, 4, 57, 76, 58, 40, 77, 18, 4, 10]
[40, 4, 57, 76, 58, 40, 77, 18, 4, 10, 27]
[40, 4, 57, 76, 58, 40, 77, 18, 4, 10, 27, 4]
[40, 4, 57, 76, 58, 40, 77, 18, 4, 10, 27, 4, 44]
[40, 4, 57, 76, 58, 40, 77, 18, 4, 10, 27, 4, 44, 40]
[40, 4, 57, 76, 58, 40, 77, 18, 4, 10, 27, 4, 44, 40, 78]
[58, 40]
[58, 40, 103]
[58, 40, 103, 1]
[58, 40, 103, 1, 267]

[58, 40, 103, 1, 267, 268]
[58, 40, 103, 1, 267, 268, 269]
[58, 40, 103, 1, 267, 268, 269, 42]
[58, 40, 103, 1, 267, 268, 269, 42, 1]
[58, 40, 103, 1, 267, 268, 269, 42, 1, 26]
[58, 40, 103, 1, 267, 268, 269, 42, 1, 26, 7]
[18, 4]
[18, 4, 10]
[18, 4, 10, 27]
[18, 4, 10, 27, 4]
[18, 4, 10, 27, 4, 155]
[18, 4, 10, 27, 4, 155, 1]
[18, 4, 10, 27, 4, 155, 1, 270]
[18, 4, 10, 27, 4, 155, 1, 270, 17]
[18, 4, 10, 27, 4, 155, 1, 270, 17, 1]
[18, 4, 10, 27, 4, 155, 1, 270, 17, 1, 38]
[18, 4, 10, 27, 4, 155, 1, 270, 17, 1, 38, 25]
[18, 4, 10, 27, 4, 155, 1, 270, 17, 1, 38, 25, 6]
[18, 4, 10, 27, 4, 155, 1, 270, 17, 1, 38, 25, 6, 1]
[18, 4, 10, 27, 4, 155, 1, 270, 17, 1, 38, 25, 6, 1, 53]
[18, 4, 10, 27, 4, 155, 1, 270, 17, 1, 38, 25, 6, 1, 53, 30]
[18, 4, 10, 27, 4, 155, 1, 270, 17, 1, 38, 25, 6, 1, 53, 30, 11]
[44, 40]
[44, 40, 9]
[44, 40, 9, 1]
[44, 40, 9, 1, 271]
[44, 40, 9, 1, 271, 42]
[44, 40, 9, 1, 271, 42, 1]
[44, 40, 9, 1, 271, 42, 1, 26]
[44, 40, 9, 1, 271, 42, 1, 26, 7]
[57, 9]
[57, 9, 1]
[57, 9, 1, 56]
[57, 9, 1, 56, 45]
[57, 9, 1, 56, 45, 39]
[26, 7]
[26, 7]
[59, 25]
[59, 25, 7]
[1, 272]
[1, 272, 17]
[1, 272, 17, 156]
[1, 272, 17, 156, 8]
[1, 272, 17, 156, 8, 5]
[1, 272, 17, 156, 8, 5, 1]
[1, 272, 17, 156, 8, 5, 1, 19]
[1, 272, 17, 156, 8, 5, 1, 19, 11]
[1, 272, 17, 156, 8, 5, 1, 19, 11, 9]
[1, 272, 17, 156, 8, 5, 1, 19, 11, 9, 148]
[1, 272, 17, 156, 8, 5, 1, 19, 11, 9, 148, 34]
[1, 272, 17, 156, 8, 5, 1, 19, 11, 9, 148, 34, 1]
[1, 272, 17, 156, 8, 5, 1, 19, 11, 9, 148, 34, 1, 25]
[1, 272, 17, 156, 8, 5, 1, 19, 11, 9, 148, 34, 1, 25, 7]
[273, 1]
[273, 1, 8]
[273, 1, 8, 9]

[273, 1, 8, 9, 157]
[273, 1, 8, 9, 157, 35]
[273, 1, 8, 9, 157, 35, 1]
[273, 1, 8, 9, 157, 35, 1, 56]
[273, 1, 8, 9, 157, 35, 1, 56, 39]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5, 43]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5, 43, 275]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5, 43, 275, 276]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5, 43, 275, 276, 5]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5, 43, 275, 276, 5, 1]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5, 43, 275, 276, 5, 1, 26]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5, 43, 275, 276, 5, 1, 26, 7]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5, 43, 275, 276, 5, 1, 26, 7, 35]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5, 43, 275, 276, 5, 1, 26, 7, 35, 277]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5, 43, 275, 276, 5, 1, 26, 7, 35, 277, 18]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5, 43, 275, 276, 5, 1, 26, 7, 35, 277, 18, 4]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5, 43, 275, 276, 5, 1, 26, 7, 35, 277, 18, 4, 10]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5, 43, 275, 276, 5, 1, 26, 7, 35, 277, 18, 4, 10, 6]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5, 43, 275, 276, 5, 1, 26, 7, 35, 277, 18, 4, 10, 6, 27]
[273, 1, 8, 9, 157, 35, 1, 56, 39, 6, 274, 1, 28, 5, 43, 275, 276, 5, 1, 26, 7, 35, 277, 18, 4, 10, 6, 27, 4]
[75, 15]
[75, 15, 158]
[75, 15, 158, 9]
[75, 15, 158, 9, 278]
[75, 15, 158, 9, 278, 35]
[75, 15, 158, 9, 278, 35, 104]
[75, 15, 158, 9, 278, 35, 104, 39]
[75, 15, 158, 9, 278, 35, 104, 39, 94]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10, 5]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10, 5, 10]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10, 5, 10, 31]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10, 5, 10, 31, 280]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10, 5, 10, 31, 280, 159]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10, 5, 10, 31, 280, 159, 1]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10, 5, 10, 31, 280,

159, 1, 281]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10, 5, 10, 31, 280, 159, 1, 281, 28]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10, 5, 10, 31, 280, 159, 1, 281, 28, 36]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10, 5, 10, 31, 280, 159, 1, 281, 28, 36, 18]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10, 5, 10, 31, 280, 159, 1, 281, 28, 36, 18, 4]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10, 5, 10, 31, 280, 159, 1, 281, 28, 36, 18, 4, 10]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10, 5, 10, 31, 280, 159, 1, 281, 28, 36, 18, 4, 10, 6]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10, 5, 10, 31, 280, 159, 1, 281, 28, 36, 18, 4, 10, 6, 27]
[75, 15, 158, 9, 278, 35, 104, 39, 94, 279, 123, 20, 36, 10, 5, 10, 31, 280, 159, 1, 281, 28, 36, 18, 4, 10, 6, 27, 4]
[160, 122]
[160, 122, 1]
[160, 122, 1, 28]
[160, 122, 1, 28, 17]
[160, 122, 1, 28, 17, 1]
[160, 122, 1, 28, 17, 1, 158]
[160, 122, 1, 28, 17, 1, 158, 6]
[160, 122, 1, 28, 17, 1, 158, 6, 1]
[160, 122, 1, 28, 17, 1, 158, 6, 1, 157]
[160, 122, 1, 28, 17, 1, 158, 6, 1, 157, 28]
[160, 122, 1, 28, 17, 1, 158, 6, 1, 157, 28, 52]
[160, 122, 1, 28, 17, 1, 158, 6, 1, 157, 28, 52, 161]
[160, 122, 1, 28, 17, 1, 158, 6, 1, 157, 28, 52, 161, 5]
[160, 122, 1, 28, 17, 1, 158, 6, 1, 157, 28, 52, 161, 5, 282]
[160, 122, 1, 28, 17, 1, 158, 6, 1, 157, 28, 52, 161, 5, 282, 1]
[160, 122, 1, 28, 17, 1, 158, 6, 1, 157, 28, 52, 161, 5, 282, 1, 156]
[160, 122, 1, 28, 17, 1, 158, 6, 1, 157, 28, 52, 161, 5, 282, 1, 156, 8]
[1, 153]
[1, 153, 16]
[1, 153, 16, 1]
[1, 153, 16, 1, 25]
[1, 153, 16, 1, 25, 7]
[1, 153, 16, 1, 25, 7, 9]
[60, 4]
[60, 4, 57]
[60, 4, 57, 76]
[60, 4, 57, 76, 58]
[60, 4, 57, 76, 58, 60]
[60, 4, 57, 76, 58, 60, 77]
[60, 4, 57, 76, 58, 60, 77, 18]
[60, 4, 57, 76, 58, 60, 77, 18, 4]
[60, 4, 57, 76, 58, 60, 77, 18, 4, 10]
[60, 4, 57, 76, 58, 60, 77, 18, 4, 10, 27]
[60, 4, 57, 76, 58, 60, 77, 18, 4, 10, 27, 4]
[60, 4, 57, 76, 58, 60, 77, 18, 4, 10, 27, 4, 44]
[60, 4, 57, 76, 58, 60, 77, 18, 4, 10, 27, 4, 44, 60]
[60, 4, 57, 76, 58, 60, 77, 18, 4, 10, 27, 4, 44, 60, 78]
[162, 12]
[162, 12, 4]

[162, 12, 4, 163]
[162, 12, 4, 163, 76]
[162, 12, 4, 163, 76, 58]
[162, 12, 4, 163, 76, 58, 12]
[162, 12, 4, 163, 76, 58, 12, 77]
[162, 12, 4, 163, 76, 58, 12, 77, 18]
[162, 12, 4, 163, 76, 58, 12, 77, 18, 4]
[162, 12, 4, 163, 76, 58, 12, 77, 18, 4, 10]
[162, 12, 4, 163, 76, 58, 12, 77, 18, 4, 10, 27]
[162, 12, 4, 163, 76, 58, 12, 77, 18, 4, 10, 27, 4]
[162, 12, 4, 163, 76, 58, 12, 77, 18, 4, 10, 27, 4, 44]
[162, 12, 4, 163, 76, 58, 12, 77, 18, 4, 10, 27, 4, 44, 12]
[162, 12, 4, 163, 76, 58, 12, 77, 18, 4, 10, 27, 4, 44, 12, 78]
[79, 283]
[79, 283, 1]
[79, 283, 1, 53]
[79, 283, 1, 53, 11]
[79, 283, 1, 53, 11, 34]
[79, 283, 1, 53, 11, 34, 40]
[79, 283, 1, 53, 11, 34, 40, 4]
[79, 283, 1, 53, 11, 34, 40, 4, 284]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103, 1]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103, 1, 144]

[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103, 1, 144, 290]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103, 1, 144, 290, 28]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103, 1, 144, 290, 28, 291]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103, 1, 144, 290, 28, 291, 34]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103, 1, 144, 290, 28, 291, 34, 292]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103, 1, 144, 290, 28, 291, 34, 292, 2
93]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103, 1, 144, 290, 28, 291, 34, 292, 2
93, 79]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103, 1, 144, 290, 28, 291, 34, 292, 2
93, 79, 164]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103, 1, 144, 290, 28, 291, 34, 292, 2
93, 79, 164, 5]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103, 1, 144, 290, 28, 291, 34, 292, 2
93, 79, 164, 5, 294]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103, 1, 144, 290, 28, 291, 34, 292, 2
93, 79, 164, 5, 294, 295]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103, 1, 144, 290, 28, 291, 34, 292, 2
93, 79, 164, 5, 294, 295, 11]
[79, 283, 1, 53, 11, 34, 40, 4, 284, 285, 286, 1, 8, 79, 287, 164, 5, 288, 1
39, 75, 79, 289, 60, 4, 46, 12, 4, 31, 103, 1, 144, 290, 28, 291, 34, 292, 2
93, 79, 164, 5, 294, 295, 11, 296]
[12, 4]
[12, 4, 40]
[12, 4, 40, 4]
[12, 4, 40, 4, 46]
[12, 4, 40, 4, 46, 12]
[12, 4, 40, 4, 46, 12, 4]
[12, 4, 40, 4, 46, 12, 4, 10]
[12, 4, 40, 4, 46, 12, 4, 10, 60]
[12, 4, 40, 4, 46, 12, 4, 10, 60, 4]
[12, 4, 40, 4, 46, 12, 4, 10, 60, 4, 46]
[12, 4, 40, 4, 46, 12, 4, 10, 60, 4, 46, 162]
[12, 4, 40, 4, 46, 12, 4, 10, 60, 4, 46, 162, 12]
[12, 4, 40, 4, 46, 12, 4, 10, 60, 4, 46, 162, 12, 4]
[46, 155]
[46, 155, 105]
[46, 155, 105, 106]
[46, 155, 105, 106, 165]
[104, 9]
[104, 9, 45]
[104, 9, 45, 39]
[25, 7]
[25, 7]

[61, 20]
[61, 20, 7]
[1, 20]
[1, 20, 7]
[1, 20, 7, 145]
[1, 20, 7, 145, 31]
[1, 20, 7, 145, 31, 8]
[1, 20, 7, 145, 31, 8, 36]
[1, 20, 7, 145, 31, 8, 36, 1]
[1, 20, 7, 145, 31, 8, 36, 1, 38]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1, 38]

[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1, 38, 25]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1, 38, 25, 27]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1, 38, 25, 27,
62]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1, 38, 25, 27,
62, 107]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1, 38, 25, 27,
62, 107, 297]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1, 38, 25, 27,
62, 107, 297, 34]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1, 38, 25, 27,
62, 107, 297, 34, 15]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1, 38, 25, 27,
62, 107, 297, 34, 15, 56]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1, 38, 25, 27,
62, 107, 297, 34, 15, 56, 39]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1, 38, 25, 27,
62, 107, 297, 34, 15, 56, 39, 5]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1, 38, 25, 27,
62, 107, 297, 34, 15, 56, 39, 5, 140]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1, 38, 25, 27,
62, 107, 297, 34, 15, 56, 39, 5, 140, 1]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1, 38, 25, 27,
62, 107, 297, 34, 15, 56, 39, 5, 140, 1, 20]
[1, 20, 7, 145, 31, 8, 36, 1, 38, 19, 11, 73, 43, 74, 97, 1, 30, 11, 20, 16
0, 1, 38, 33, 166, 51, 88, 1, 53, 30, 11, 18, 4, 10, 107, 6, 1, 38, 25, 27,
62, 107, 297, 34, 15, 56, 39, 5, 140, 1, 20, 141]
[47, 4]
[47, 4, 57]
[47, 4, 57, 76]
[47, 4, 57, 76, 58]
[47, 4, 57, 76, 58, 47]
[47, 4, 57, 76, 58, 47, 77]
[47, 4, 57, 76, 58, 47, 77, 18]
[47, 4, 57, 76, 58, 47, 77, 18, 4]
[47, 4, 57, 76, 58, 47, 77, 18, 4, 10]
[47, 4, 57, 76, 58, 47, 77, 18, 4, 10, 27]
[47, 4, 57, 76, 58, 47, 77, 18, 4, 10, 27, 4]
[47, 4, 57, 76, 58, 47, 77, 18, 4, 10, 27, 4, 44]
[47, 4, 57, 76, 58, 47, 77, 18, 4, 10, 27, 4, 44, 47]
[47, 4, 57, 76, 58, 47, 77, 18, 4, 10, 27, 4, 44, 47, 78]
[102, 1]
[102, 1, 38]

```
[102, 1, 38, 19]
[102, 1, 38, 19, 11]
[102, 1, 38, 19, 11, 12]
[102, 1, 38, 19, 11, 12, 62]
[102, 1, 38, 19, 11, 12, 62, 9]
[102, 1, 38, 19, 11, 12, 62, 9, 74]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19, 11]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19, 11, 9]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19, 11, 9, 161]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19, 11, 9, 161, 105]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19, 11, 9, 161, 105, 106]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19, 11, 9, 161, 105, 106, 42]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19, 11, 9, 161, 105, 106, 42, 47]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19, 11, 9, 161, 105, 106, 42, 47, 62]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19, 11, 9, 161, 105, 106, 42, 47, 62, 5]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19, 11, 9, 161, 105, 106, 42, 47, 62, 5, 302]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19, 11, 9, 161, 105, 106, 42, 47, 62, 5, 302, 1]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19, 11, 9, 161, 105, 106, 42, 47, 62, 5, 302, 1, 30]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19, 11, 9, 161, 105, 106, 42, 47, 62, 5, 302, 1, 30,
11]
```

[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19, 11, 9, 161, 105, 106, 42, 47, 62, 5, 302, 1, 30,
11, 18]
[102, 1, 38, 19, 11, 12, 62, 9, 74, 98, 15, 104, 45, 5, 298, 299, 28, 149, 1
0, 6, 10, 300, 55, 301, 19, 11, 9, 161, 105, 106, 42, 47, 62, 5, 302, 1, 30,
11, 18, 4]
[18, 4]
[18, 4, 47]
[18, 4, 47, 4]
[18, 4, 47, 4, 46]
[18, 4, 47, 4, 46, 163]
[18, 4, 47, 4, 46, 163, 12]
[18, 4, 47, 4, 46, 163, 12, 4]
[47, 62]
[47, 62, 9]
[47, 62, 9, 1]
[47, 62, 9, 1, 20]
[47, 62, 9, 1, 20, 7]
[47, 62, 9, 1, 20, 7, 45]
[12, 62]
[12, 62, 9]
[12, 62, 9, 1]
[12, 62, 9, 1, 38]
[12, 62, 9, 1, 38, 19]
[12, 62, 9, 1, 38, 19, 11]
[46, 103]
[46, 103, 105]
[46, 103, 105, 106]
[46, 103, 105, 106, 165]
[57, 9]
[57, 9, 1]
[57, 9, 1, 56]
[57, 9, 1, 56, 45]
[57, 9, 1, 56, 45, 39]
[55, 30]
[55, 30, 11]
[55, 30, 11, 18]
[55, 30, 11, 18, 4]
[55, 30, 11, 18, 4, 107]
[55, 30, 11, 18, 4, 107, 9]
[55, 30, 11, 18, 4, 107, 9, 75]
[55, 30, 11, 18, 4, 107, 9, 75, 74]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1, 102]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1, 102, 33]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1, 102, 33, 166]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1, 102, 33, 166, 6]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1, 102, 33, 166, 6, 96]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1, 102, 33, 166, 6, 96, 304]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1, 102, 33, 166, 6, 96, 304, 43]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1, 102, 33, 166, 6, 96, 304, 43, 130]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1, 102, 33, 166, 6, 96, 304, 43, 130,
16]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1, 102, 33, 166, 6, 96, 304, 43, 130,
16, 305]

[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1, 102, 33, 166, 6, 96, 304, 43, 130, 16, 305, 1]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1, 102, 33, 166, 6, 96, 304, 43, 130, 16, 305, 1, 20]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1, 102, 33, 166, 6, 96, 304, 43, 130, 16, 305, 1, 20, 17]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1, 102, 33, 166, 6, 96, 304, 43, 130, 16, 305, 1, 20, 17, 1]
[55, 30, 11, 18, 4, 107, 9, 75, 74, 5, 1, 102, 33, 166, 6, 96, 304, 43, 130, 16, 305, 1, 20, 17, 1, 49]
[20, 7]
[20, 7]
[131, 306]
[131, 306, 16]
[131, 306, 16, 84]
[131, 306, 16, 84, 92]
[131, 306, 16, 84, 92, 6]
[131, 306, 16, 84, 92, 6, 108]
[131, 306, 16, 84, 92, 6, 108, 307]
[132, 93]
[132, 93, 16]
[132, 93, 16, 308]
[132, 93, 16, 308, 309]
[132, 93, 16, 308, 309, 310]
[132, 93, 16, 308, 309, 310, 108]
[33, 133]
[33, 133, 134]
[33, 133, 134, 16]
[33, 133, 134, 16, 311]
[33, 133, 134, 16, 311, 312]
[33, 133, 134, 16, 311, 312, 313]
[33, 133, 134, 16, 311, 312, 313, 6]
[33, 133, 134, 16, 311, 312, 313, 6, 314]
[33, 133, 134, 16, 311, 312, 313, 6, 314, 315]
[316, 109]
[316, 109, 16]
[316, 109, 16, 317]
[316, 109, 16, 317, 6]
[316, 109, 16, 317, 6, 318]
[316, 109, 16, 317, 6, 318, 109]
[319, 320]
[319, 320, 16]
[319, 320, 16, 321]
[319, 320, 16, 321, 322]
[323, 110]
[323, 110, 16]
[323, 110, 16, 324]
[323, 110, 16, 324, 93]
[323, 110, 16, 324, 93, 6]
[323, 110, 16, 324, 93, 6, 325]
[323, 110, 16, 324, 93, 6, 325, 167]
[167, 17]
[167, 17, 14]
[167, 17, 14, 111]
[326, 327]
[326, 327, 328]

[326, 327, 328, 329]
[330, 331]
[29, 168]
[332, 9]
[332, 9, 1]
[332, 9, 1, 142]
[332, 9, 1, 142, 333]
[332, 9, 1, 142, 333, 17]
[332, 9, 1, 142, 333, 17, 35]
[332, 9, 1, 142, 333, 17, 35, 14]
[332, 9, 1, 142, 333, 17, 35, 14, 111]
[332, 9, 1, 142, 333, 17, 35, 14, 111, 71]
[332, 9, 1, 142, 333, 17, 35, 14, 111, 71, 125]
[332, 9, 1, 142, 333, 17, 35, 14, 111, 71, 125, 90]
[41, 334]
[41, 334, 335]
[41, 334, 335, 336]
[41, 334, 335, 336, 337]
[41, 334, 335, 336, 337, 95]
[41, 152]
[41, 152, 8]
[41, 152, 8, 338]
[41, 152, 8, 338, 126]
[41, 152, 8, 338, 126, 127]
[41, 339]
[41, 339, 15]
[41, 339, 15, 340]
[41, 339, 15, 340, 49]
[41, 339, 15, 340, 49, 99]
[41, 52]
[41, 52, 70]
[41, 52, 70, 342]
[41, 52, 70, 342, 16]
[41, 52, 70, 342, 16, 343]
[41, 52, 70, 342, 16, 343, 110]
[345, 5]
[345, 5, 346]
[345, 5, 346, 347]
[10, 29]
[350, 351]
[112, 66]
[22, 2]
[22, 2, 3]
[113, 170]
[113, 170, 80]
[113, 170, 80, 170]
[113, 170, 80, 170, 353]
[61, 2]
[61, 2, 3]
[61, 2]
[61, 2, 3]
[171, 2]
[171, 2, 3]
[22, 2]
[22, 2, 3]
[86, 111]

[356, 2]
[356, 2, 3]
[22, 2]
[22, 2, 3]
[358, 32]
[358, 32, 359]
[29, 2]
[29, 2, 3]
[45, 360]
[81, 2]
[81, 2, 3]
[361, 86]
[361, 86, 49]
[81, 2]
[81, 2, 3]
[171, 2]
[171, 2, 3]
[112, 66]
[112, 66, 363]
[22, 2]
[22, 2, 3]
[172, 2]
[172, 2, 3]
[29, 2]
[29, 2, 3]
[22, 2]
[22, 2, 3]
[22, 2]
[22, 2, 3]
[81, 2]
[81, 2, 3]
[172, 2]
[172, 2, 3]
[37, 369]
[72, 370]
[371, 2]
[371, 2, 3]
[372, 373]
[372, 373, 72]
[372, 373, 72, 173]
[61, 2]
[61, 2, 3]
[81, 2]
[81, 2, 3]
[29, 2]
[29, 2, 3]
[376, 173]
[22, 2]
[22, 2, 3]
[112, 66]
[112, 66, 377]
[59, 2]
[59, 2, 3]
[22, 2]
[22, 2, 3]
[381, 2]

[381, 2, 3]
[382, 383]
[29, 2]
[29, 2, 3]
[29, 2]
[29, 2, 3]
[386, 387]
[386, 387, 109]
[29, 2]
[29, 2, 3]
[388, 389]
[388, 389, 390]
[22, 2]
[22, 2, 3]
[391, 110]
[61, 2]
[61, 2, 3]
[108, 392]
[108, 392, 35]
[108, 392, 35, 14]
[29, 2]
[29, 2, 3]
[84, 92]
[22, 2]
[22, 2, 3]
[64, 168]
[393, 2]
[393, 2, 3]
[23, 48]
[23, 48, 6]
[23, 48, 6, 80]
[23, 48, 6, 80, 114]
[59, 2]
[59, 2, 3]
[23, 394]
[23, 394, 114]
[59, 2]
[59, 2, 3]
[395, 113]
[395, 113, 114]
[59, 2]
[59, 2, 3]
[115, 396]
[115, 396, 175]
[15, 397]
[15, 397, 398]
[15, 397, 398, 399]
[15, 397, 398, 399, 400]
[15, 397, 398, 399, 400, 115]
[15, 397, 398, 399, 400, 115, 176]
[15, 397, 398, 399, 400, 115, 176, 177]
[15, 397, 398, 399, 400, 115, 176, 177, 401]
[15, 397, 398, 399, 400, 115, 176, 177, 401, 178]
[15, 397, 398, 399, 400, 115, 176, 177, 401, 178, 179]
[15, 397, 398, 399, 400, 115, 176, 177, 401, 178, 179, 180]
[15, 397, 398, 399, 400, 115, 176, 177, 401, 178, 179, 180, 181]

[402, 175]
[182, 403]
[182, 403, 176]
[182, 403, 176, 182]
[182, 403, 176, 182, 404]
[182, 403, 176, 182, 404, 405]
[182, 403, 176, 182, 404, 405, 406]
[182, 403, 176, 182, 404, 405, 406, 177]
[182, 403, 176, 182, 404, 405, 406, 177, 407]
[182, 403, 176, 182, 404, 405, 406, 177, 407, 178]
[182, 403, 176, 182, 404, 405, 406, 177, 407, 178, 408]
[182, 403, 176, 182, 404, 405, 406, 177, 407, 178, 408, 409]
[182, 403, 176, 182, 404, 405, 406, 177, 407, 178, 408, 409, 410]
[182, 403, 176, 182, 404, 405, 406, 177, 407, 178, 408, 409, 410, 179]
[182, 403, 176, 182, 404, 405, 406, 177, 407, 178, 408, 409, 410, 179, 180]
[182, 403, 176, 182, 404, 405, 406, 177, 407, 178, 408, 409, 410, 179, 180, 181]
[82, 116]
[82, 116, 183]
[82, 116, 183, 411]
[82, 116, 183, 411, 89]
[82, 116]
[82, 116, 183]
[82, 116, 183, 116]
[82, 116, 183, 116, 89]
[413, 117]
[415, 416]
[417, 117]
[418, 42]
[418, 42, 117]
[82, 115]
[82, 115, 419]
[420, 95]
[420, 95, 421]
[423, 15]
[423, 15, 424]
[426, 427]
[426, 427, 428]
[184, 185]
[118, 429]
[113, 80]
[113, 80, 23]
[113, 80, 23, 48]
[186, 430]
[186, 430, 187]
[64, 188]
[431, 6]
[431, 6, 432]
[80, 6]
[80, 6, 23]
[80, 6, 23, 48]
[65, 6]
[65, 6, 433]
[118, 189]
[184, 185]
[85, 434]

```
[435, 436]
[118, 189]
[23, 48]
[186, 187]
[188, 190]
[64, 190]
[82, 440]
[441, 442]
[83, 443]
[83, 443, 444]
[83, 443, 444, 445]
[83, 443, 444, 445, 446]
[83, 443, 444, 445, 446, 159]
[83, 443, 444, 445, 446, 159, 447]
[83, 443, 444, 445, 446, 159, 447, 448]
```

This is input and output column (last number is uoutput rest is input)

- A joint List $n \times 2$
- n is number of word in each sentence

This is not perfect for LSTM

- why? -> Each sentence have different in size But in LSTM takes Same size vector
 - so we resize then and make same dimensional vector
- table>

Tokenized Dataset

Input	Output
My	name
My name	is
My name is	Srabon

Resized Tokenized Dataset

Input(extra space must be fill by 0 prefix)	Output
[0,0,1]	[2]
[0,1,2]	[3]
[1,2,3]	[4]

```
In [11]: mx=0
         for i in input_sequences:
             mx=max(mx, len(i)) # Finding the maximum length of the n-gram sequences
```

```
In [12]: mx
```


[0,0,1]	[0,1,0,0] (2)
[0,1,2]	[0,0,1,0](3)
[1,2,3]	[0,0,0,1](4)

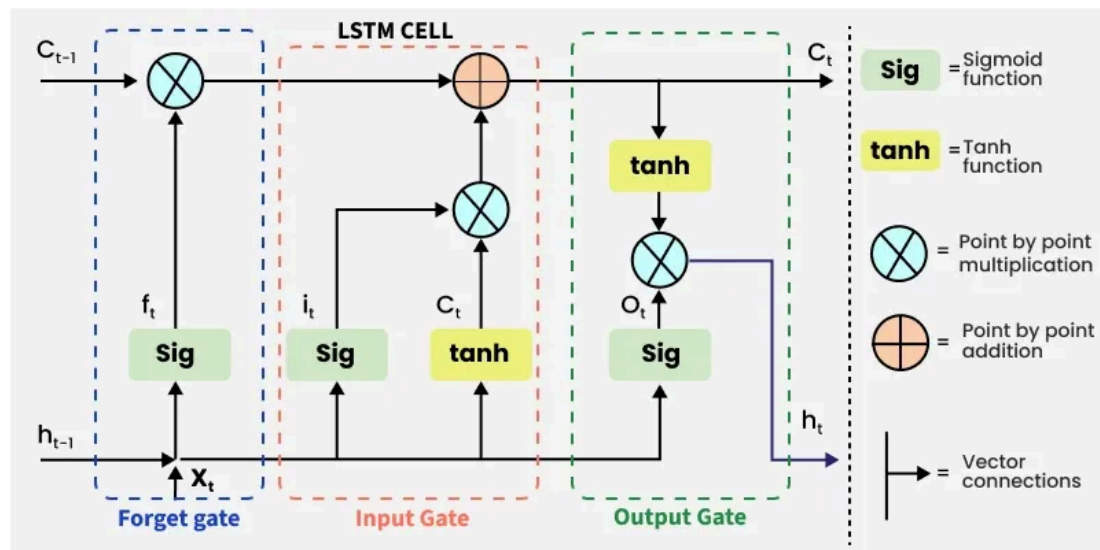
```
In [18]: import numpy as np
import tensorflow as tf
from tensorflow.keras.utils import to_categorical
Output=to_categorical(Output, num_classes=len(tokenizer.word_index)+1) # Co
```

```
In [19]: X
```

```
Out[19]: array([[ 0,  0,  0, ...,  0,  0, 64],
 [ 0,  0,  0, ...,  0,  0, 192],
 [ 0,  0,  0, ...,  0,  0, 23],
 ...,
 [ 0,  0,  0, ..., 444, 445, 446],
 [ 0,  0,  0, ..., 445, 446, 159],
 [ 0,  0,  0, ..., 446, 159, 447]], shape=(1056, 50), dtype=int32)
```

Input and Output creation done perfectly now we go to LSTM architecture

Core Conceptual Architecture of LSTM



```
In [20]: from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Embedding, LSTM, Dense

model = Sequential([
    Input(shape=(50,)), # Only sequence length
    Embedding(input_dim=449, output_dim=100),
    LSTM(150),
    Dense(449, activation="softmax")
])
```

```
model.summary()
```

```
E0000 00:00:1783975228.343307 525085 cuda_platform.cc:52] failed call to cu
Init: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)
```

Model: "sequential"

Layer (type)	Output Shape	Par
embedding (Embedding)	(None, 50, 100)	44
lstm (LSTM)	(None, 150)	150
dense (Dense)	(None, 449)	67

Total params: 263,299 (1.00 MB)

Trainable params: 263,299 (1.00 MB)

Non-trainable params: 0 (0.00 B)

[

$$\text{Input Tokens} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 2 \\ 1 & 2 & 3 \end{bmatrix}$$

$$\text{Embedding Matrix } E = \begin{bmatrix} 0.10 & 0.20 & 0.30 & 0.40 & 0.50 \\ 0.60 & 0.70 & 0.80 & 0.90 & 1.00 \\ 1.10 & 1.20 & 1.30 & 1.40 & 1.50 \\ 1.60 & 1.70 & 1.80 & 1.90 & 2.00 \end{bmatrix}$$

$$\text{Embedded Output} = \begin{bmatrix} \begin{bmatrix} 0.10 & 0.20 & 0.30 & 0.40 & 0.50 \\ 0.10 & 0.20 & 0.30 & 0.40 & 0.50 \\ 0.60 & 0.70 & 0.80 & 0.90 & 1.00 \end{bmatrix} \\ \begin{bmatrix} 0.10 & 0.20 & 0.30 & 0.40 & 0.50 \\ 0.60 & 0.70 & 0.80 & 0.90 & 1.00 \\ 1.10 & 1.20 & 1.30 & 1.40 & 1.50 \end{bmatrix} \\ \begin{bmatrix} 0.60 & 0.70 & 0.80 & 0.90 & 1.00 \\ 1.10 & 1.20 & 1.30 & 1.40 & 1.50 \\ 1.60 & 1.70 & 1.80 & 1.90 & 2.00 \end{bmatrix} \end{bmatrix}$$

]

```
In [21]: model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['ac
```

```
In [23]: model.fit(X, Output, epochs=100, batch_size=32) # Training the model using
```

Epoch 1/100	
33/33  6s 93ms/step - accuracy: 0.0455 - loss: 5.7833	
Epoch 2/100	
33/33  3s 89ms/step - accuracy: 0.0597 - loss: 5.3011	
Epoch 3/100	
33/33  3s 83ms/step - accuracy: 0.0805 - loss: 5.1522	
Epoch 4/100	
33/33  3s 90ms/step - accuracy: 0.1089 - loss: 5.0027	
Epoch 5/100	
33/33  3s 90ms/step - accuracy: 0.1136 - loss: 4.8598	
Epoch 6/100	
33/33  3s 84ms/step - accuracy: 0.1212 - loss: 4.7281	
Epoch 7/100	
33/33  4s 114ms/step - accuracy: 0.1439 - loss: 4.5868	
Epoch 8/100	
33/33  3s 99ms/step - accuracy: 0.1629 - loss: 4.4029	
Epoch 9/100	
33/33  4s 107ms/step - accuracy: 0.1884 - loss: 4.2023	
Epoch 10/100	
33/33  6s 128ms/step - accuracy: 0.2074 - loss: 4.0044	
Epoch 11/100	
33/33  4s 113ms/step - accuracy: 0.2254 - loss: 3.8000	
Epoch 12/100	
33/33  4s 112ms/step - accuracy: 0.2614 - loss: 3.6066	
Epoch 13/100	
33/33  4s 110ms/step - accuracy: 0.3002 - loss: 3.4138	
Epoch 14/100	
33/33  5s 146ms/step - accuracy: 0.3258 - loss: 3.2417	
Epoch 15/100	
33/33  5s 130ms/step - accuracy: 0.3551 - loss: 3.0695	
Epoch 16/100	
33/33  3s 91ms/step - accuracy: 0.3892 - loss: 2.8921	
Epoch 17/100	
33/33  3s 87ms/step - accuracy: 0.4025 - loss: 2.7402	
Epoch 18/100	
33/33  4s 110ms/step - accuracy: 0.4441 - loss: 2.5940	
Epoch 19/100	
33/33  4s 133ms/step - accuracy: 0.4640 - loss: 2.4453	
Epoch 20/100	
33/33  5s 122ms/step - accuracy: 0.4867 - loss: 2.3093	
Epoch 21/100	
33/33  4s 121ms/step - accuracy: 0.5369 - loss: 2.1807	
Epoch 22/100	
33/33  4s 115ms/step - accuracy: 0.5824 - loss: 2.0501	
Epoch 23/100	
33/33  4s 124ms/step - accuracy: 0.6089 - loss: 1.9368	
Epoch 24/100	
33/33  4s 126ms/step - accuracy: 0.6354 - loss: 1.8214	
Epoch 25/100	
33/33  4s 122ms/step - accuracy: 0.6761 - loss: 1.7062	
Epoch 26/100	
33/33  4s 124ms/step - accuracy: 0.7008 - loss: 1.6049	
Epoch 27/100	
33/33  4s 114ms/step - accuracy: 0.7225 - loss: 1.5045	
Epoch 28/100	
33/33  4s 117ms/step - accuracy: 0.7462 - loss: 1.4142	

Epoch 29/100			
33/33		4s 112ms/step	- accuracy: 0.7737 - loss: 1.3233
Epoch 30/100			
33/33		4s 127ms/step	- accuracy: 0.7926 - loss: 1.2369
Epoch 31/100			
33/33		4s 125ms/step	- accuracy: 0.8078 - loss: 1.1597
Epoch 32/100			
33/33		4s 129ms/step	- accuracy: 0.8314 - loss: 1.0868
Epoch 33/100			
33/33		5s 124ms/step	- accuracy: 0.8419 - loss: 1.0250
Epoch 34/100			
33/33		4s 127ms/step	- accuracy: 0.8617 - loss: 0.9553
Epoch 35/100			
33/33		4s 128ms/step	- accuracy: 0.8712 - loss: 0.8921
Epoch 36/100			
33/33		4s 128ms/step	- accuracy: 0.8835 - loss: 0.8395
Epoch 37/100			
33/33		4s 126ms/step	- accuracy: 0.8949 - loss: 0.7817
Epoch 38/100			
33/33		5s 139ms/step	- accuracy: 0.9062 - loss: 0.7357
Epoch 39/100			
33/33		5s 136ms/step	- accuracy: 0.9034 - loss: 0.6906
Epoch 40/100			
33/33		5s 136ms/step	- accuracy: 0.9176 - loss: 0.6509
Epoch 41/100			
33/33		5s 138ms/step	- accuracy: 0.9242 - loss: 0.6108
Epoch 42/100			
33/33		5s 138ms/step	- accuracy: 0.9290 - loss: 0.5773
Epoch 43/100			
33/33		5s 134ms/step	- accuracy: 0.9299 - loss: 0.5414
Epoch 44/100			
33/33		4s 131ms/step	- accuracy: 0.9318 - loss: 0.5116
Epoch 45/100			
33/33		4s 127ms/step	- accuracy: 0.9328 - loss: 0.4828
Epoch 46/100			
33/33		4s 130ms/step	- accuracy: 0.9394 - loss: 0.4580
Epoch 47/100			
33/33		4s 133ms/step	- accuracy: 0.9413 - loss: 0.4347
Epoch 48/100			
33/33		4s 132ms/step	- accuracy: 0.9432 - loss: 0.4128
Epoch 49/100			
33/33		4s 125ms/step	- accuracy: 0.9451 - loss: 0.3886
Epoch 50/100			
33/33		5s 138ms/step	- accuracy: 0.9451 - loss: 0.3694
Epoch 51/100			
33/33		4s 127ms/step	- accuracy: 0.9470 - loss: 0.3538
Epoch 52/100			
33/33		4s 129ms/step	- accuracy: 0.9470 - loss: 0.3373
Epoch 53/100			
33/33		4s 129ms/step	- accuracy: 0.9498 - loss: 0.3211
Epoch 54/100			
33/33		4s 132ms/step	- accuracy: 0.9508 - loss: 0.3092
Epoch 55/100			
33/33		4s 134ms/step	- accuracy: 0.9489 - loss: 0.2975
Epoch 56/100			
33/33		4s 124ms/step	- accuracy: 0.9498 - loss: 0.2821

Epoch 57/100	
33/33  4s 128ms/step - accuracy: 0.9536 - loss: 0.2710	
Epoch 58/100	
33/33  4s 132ms/step - accuracy: 0.9517 - loss: 0.2601	
Epoch 59/100	
33/33  4s 129ms/step - accuracy: 0.9555 - loss: 0.2512	
Epoch 60/100	
33/33  4s 127ms/step - accuracy: 0.9508 - loss: 0.2442	
Epoch 61/100	
33/33  6s 151ms/step - accuracy: 0.9545 - loss: 0.2345	
Epoch 62/100	
33/33  4s 132ms/step - accuracy: 0.9602 - loss: 0.2264	
Epoch 63/100	
33/33  4s 124ms/step - accuracy: 0.9517 - loss: 0.2192	
Epoch 64/100	
33/33  4s 123ms/step - accuracy: 0.9527 - loss: 0.2140	
Epoch 65/100	
33/33  4s 123ms/step - accuracy: 0.9574 - loss: 0.2047	
Epoch 66/100	
33/33  4s 126ms/step - accuracy: 0.9536 - loss: 0.2000	
Epoch 67/100	
33/33  4s 126ms/step - accuracy: 0.9536 - loss: 0.1932	
Epoch 68/100	
33/33  4s 129ms/step - accuracy: 0.9536 - loss: 0.1899	
Epoch 69/100	
33/33  5s 123ms/step - accuracy: 0.9564 - loss: 0.1840	
Epoch 70/100	
33/33  4s 129ms/step - accuracy: 0.9593 - loss: 0.1781	
Epoch 71/100	
33/33  4s 126ms/step - accuracy: 0.9564 - loss: 0.1742	
Epoch 72/100	
33/33  4s 128ms/step - accuracy: 0.9564 - loss: 0.1698	
Epoch 73/100	
33/33  4s 126ms/step - accuracy: 0.9564 - loss: 0.1658	
Epoch 74/100	
33/33  4s 125ms/step - accuracy: 0.9602 - loss: 0.1626	
Epoch 75/100	
33/33  4s 136ms/step - accuracy: 0.9593 - loss: 0.1592	
Epoch 76/100	
33/33  4s 126ms/step - accuracy: 0.9602 - loss: 0.1545	
Epoch 77/100	
33/33  4s 134ms/step - accuracy: 0.9602 - loss: 0.1505	
Epoch 78/100	
33/33  4s 123ms/step - accuracy: 0.9602 - loss: 0.1472	
Epoch 79/100	
33/33  4s 122ms/step - accuracy: 0.9583 - loss: 0.1444	
Epoch 80/100	
33/33  5s 124ms/step - accuracy: 0.9631 - loss: 0.1412	
Epoch 81/100	
33/33  5s 126ms/step - accuracy: 0.9593 - loss: 0.1378	
Epoch 82/100	
33/33  4s 127ms/step - accuracy: 0.9602 - loss: 0.1365	
Epoch 83/100	
33/33  4s 125ms/step - accuracy: 0.9631 - loss: 0.1323	
Epoch 84/100	
33/33  5s 136ms/step - accuracy: 0.9612 - loss: 0.1305	


```

Epoch 85/100
33/33  5s 154ms/step - accuracy: 0.9631 - loss: 0.1281
Epoch 86/100
33/33  8s 83ms/step - accuracy: 0.9640 - loss: 0.1266
Epoch 87/100
33/33  3s 83ms/step - accuracy: 0.9678 - loss: 0.1225
Epoch 88/100
33/33  3s 84ms/step - accuracy: 0.9602 - loss: 0.1210
Epoch 89/100
33/33  3s 82ms/step - accuracy: 0.9621 - loss: 0.1192
Epoch 90/100
33/33  6s 123ms/step - accuracy: 0.9612 - loss: 0.1176
Epoch 91/100
33/33  5s 122ms/step - accuracy: 0.9602 - loss: 0.1166
Epoch 92/100
33/33  4s 121ms/step - accuracy: 0.9650 - loss: 0.1143
Epoch 93/100
33/33  4s 120ms/step - accuracy: 0.9612 - loss: 0.1118
Epoch 94/100
33/33  4s 128ms/step - accuracy: 0.9602 - loss: 0.1114
Epoch 95/100
33/33  4s 121ms/step - accuracy: 0.9659 - loss: 0.1114
Epoch 96/100
33/33  4s 126ms/step - accuracy: 0.9621 - loss: 0.1087
Epoch 97/100
33/33  5s 122ms/step - accuracy: 0.9640 - loss: 0.1059
Epoch 98/100
33/33  5s 130ms/step - accuracy: 0.9631 - loss: 0.1048
Epoch 99/100
33/33  4s 124ms/step - accuracy: 0.9621 - loss: 0.1044
Epoch 100/100
33/33  4s 122ms/step - accuracy: 0.9612 - loss: 0.1024

```

Out[23]: <keras.src.callbacks.history.History at 0x7f83154367d0>

```

In [24]: test_text="architecture"

for i in range(10):
    token_text=tokenizer.texts_to_sequences([test_text])[0]
    padded_token=pad_sequences([token_text], maxlen=50)
    predicted=model.predict(padded_token)
    ind= np.argmax(predicted)
    for word,index in tokenizer.word_index.items():
        if index==ind:
            test_text=test_text+" "+word
            print(test_text) # Printing the generated text after predicting

```

```

1/1 _____ 0s 314ms/step
architecture is
1/1 _____ 0s 77ms/step
architecture is activation
1/1 _____ 0s 74ms/step
architecture is activation function
1/1 _____ 0s 58ms/step
architecture is activation function to
1/1 _____ 0s 93ms/step
architecture is activation function to the
1/1 _____ 0s 50ms/step
architecture is activation function to the memory
1/1 _____ 0s 64ms/step
architecture is activation function to the memory cell
1/1 _____ 0s 76ms/step
architecture is activation function to the memory cell to
1/1 _____ 0s 78ms/step
architecture is activation function to the memory cell to the
1/1 _____ 0s 54ms/step
architecture is activation function to the memory cell to the memory

```

```
In [25]: model.save("LSTM_model.h5") # Saving the trained model to a file named "LSTM_model.h5"
model.save("LSTM_model.keras")
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

At the last cell architecture find the next word and add it to the test_text variable. Then print the updated test_text variable. Each paragraph is separated by a new line character "\n".

This notebook is designed as an educational walkthrough that explains not only *what* happens inside an LSTM-based next-word prediction model, but also *why* each step is necessary.